Soriano Barrera María Elena
No. cuenta: 319154917
14 de octubre de 2026

Práctica 5. Aplicación con narrativa

Materiales y herramientas

- MacBook Air con macOS y Unity 2022.3.62f3 LTS.
- Vuforia Engine; ARCamera e Image Targets en una escena Unity.
- Visual Studio Code y scripts en C#.
- Xcode para firma y preparación de la compilación iOS.
- Cinco imágenes impresas como marcadores; webcam para validación.
- iPhone como objeto principal; cinco alternativas: busto, avión, tren, pirámide y cámara.

1. Recursos 3D e integración inicial

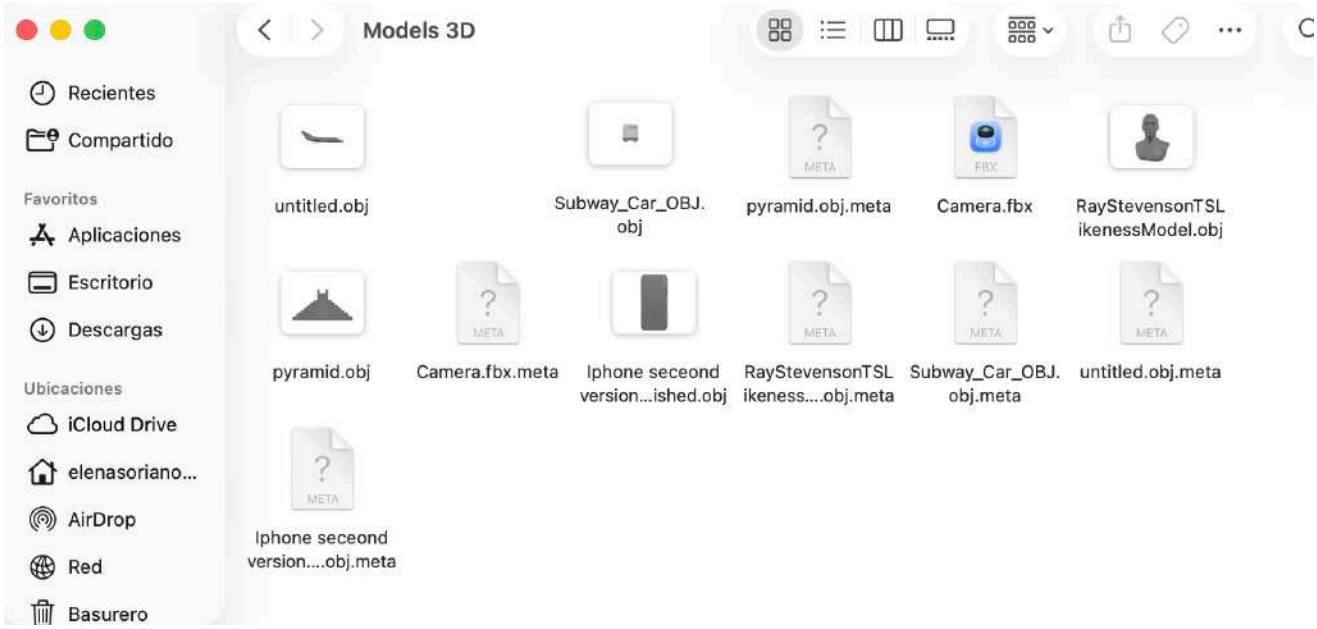


Figura 1. Carpeta de recursos originales

La captura de Finder presenta el directorio «Models 3D» con archivos de modelos OBJ y otros formatos, además de archivos .meta. Esta etapa permitió identificar los recursos que posteriormente se incorporaron a Assets, manteniendo una ubicación reconocible durante la configuración de cada Image Target.

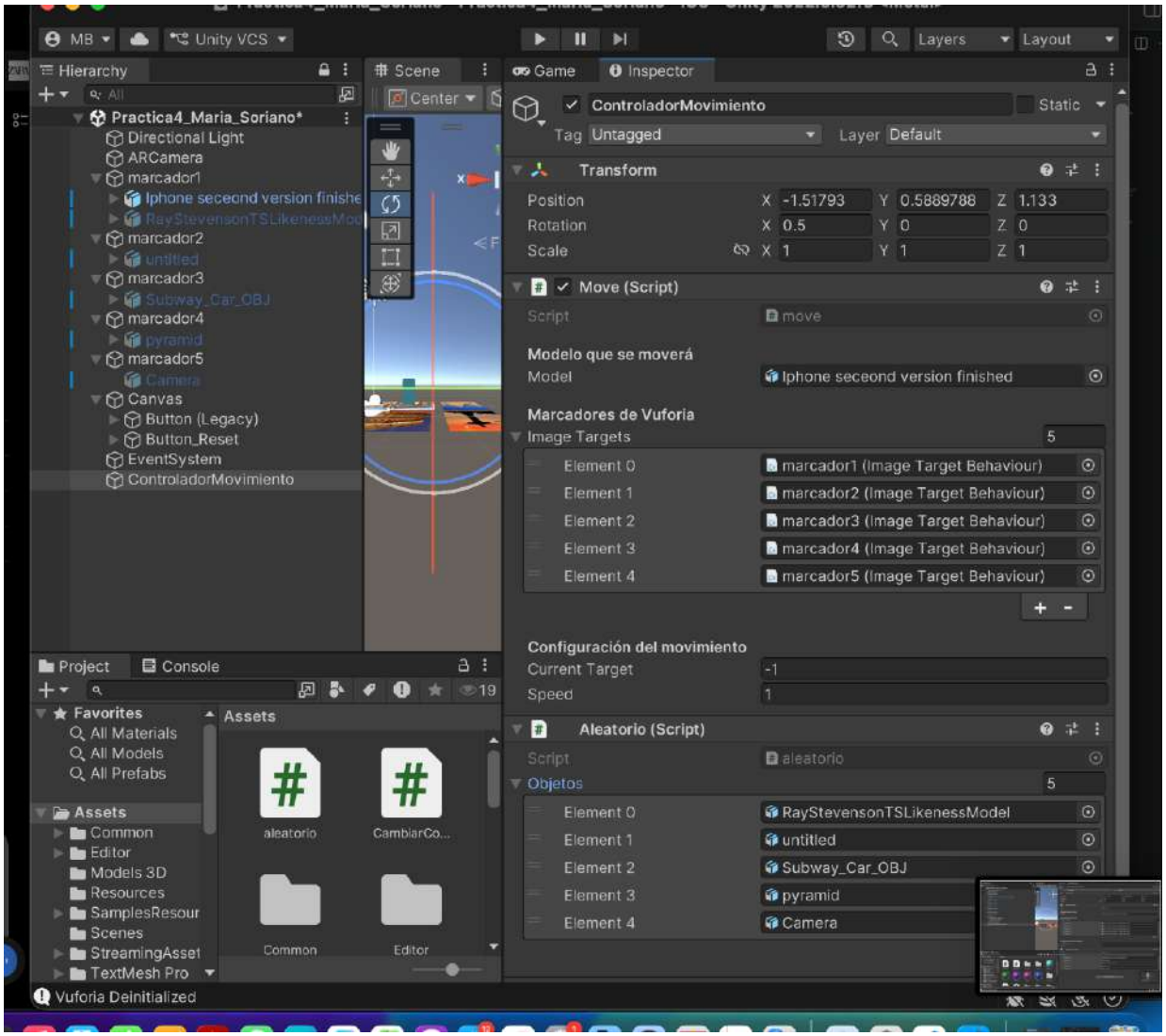


Figura 2. Jerarquía de Unity y referencias de los cinco marcadores

En Hierarchy aparecen ARCamera, marcador1–marcador5 y el controlador. En Inspector se observan las referencias a los cinco componentes Image Target Behaviour. Este arreglo es esencial para que el script recorra los marcadores y no dependa de encontrar objetos por su nombre durante la ejecución.

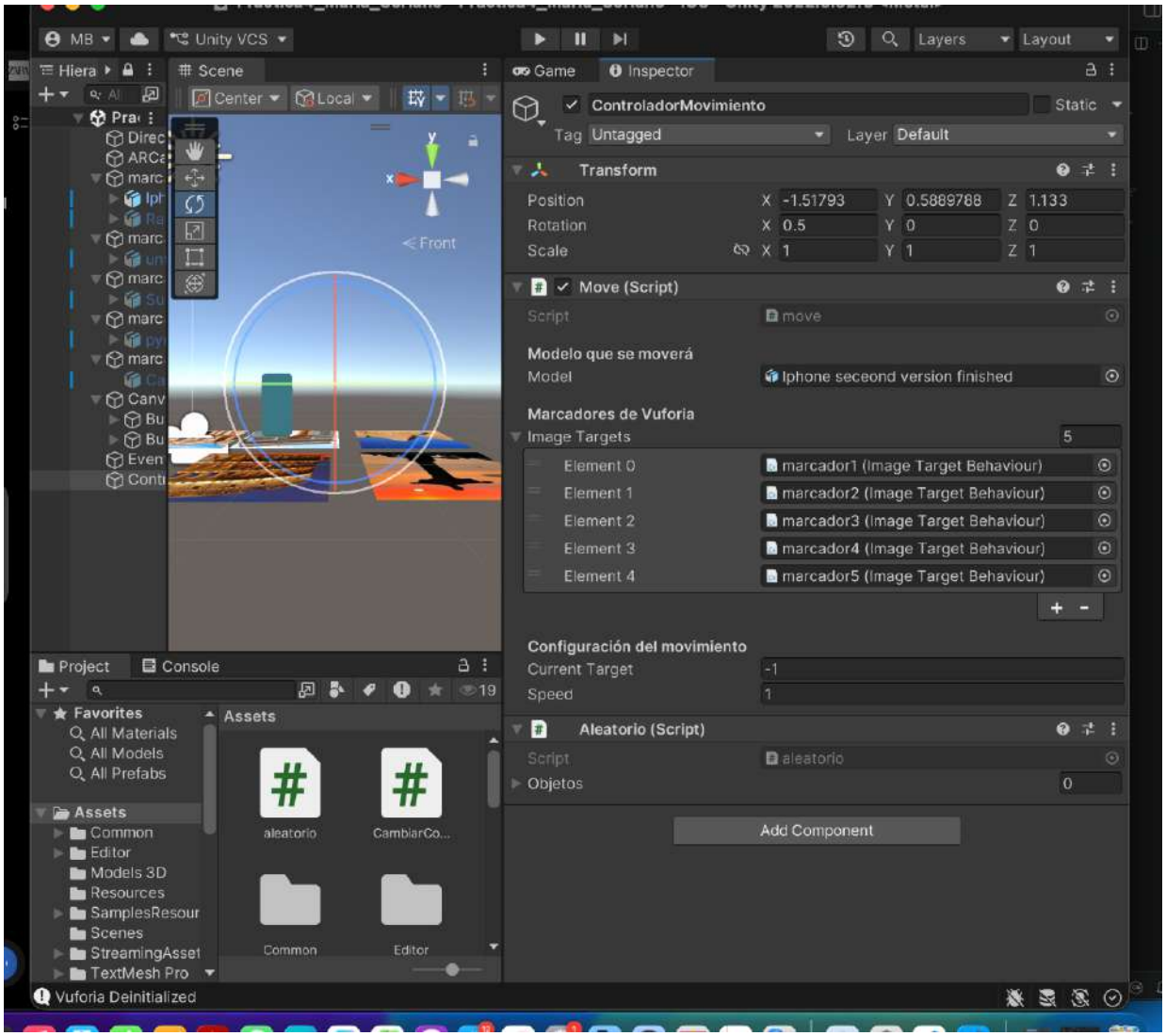


Figura 3. Estado inicial de los objetos de la escena

El modelo principal de iPhone permanece activo en marcador1, mientras las cinco alternativas 3D se dejan inicialmente desactivadas. Se evita así mostrar al inicio varias piezas superpuestas; estas alternativas se utilizan como originales para instanciar temporalmente el modelo sorteado.

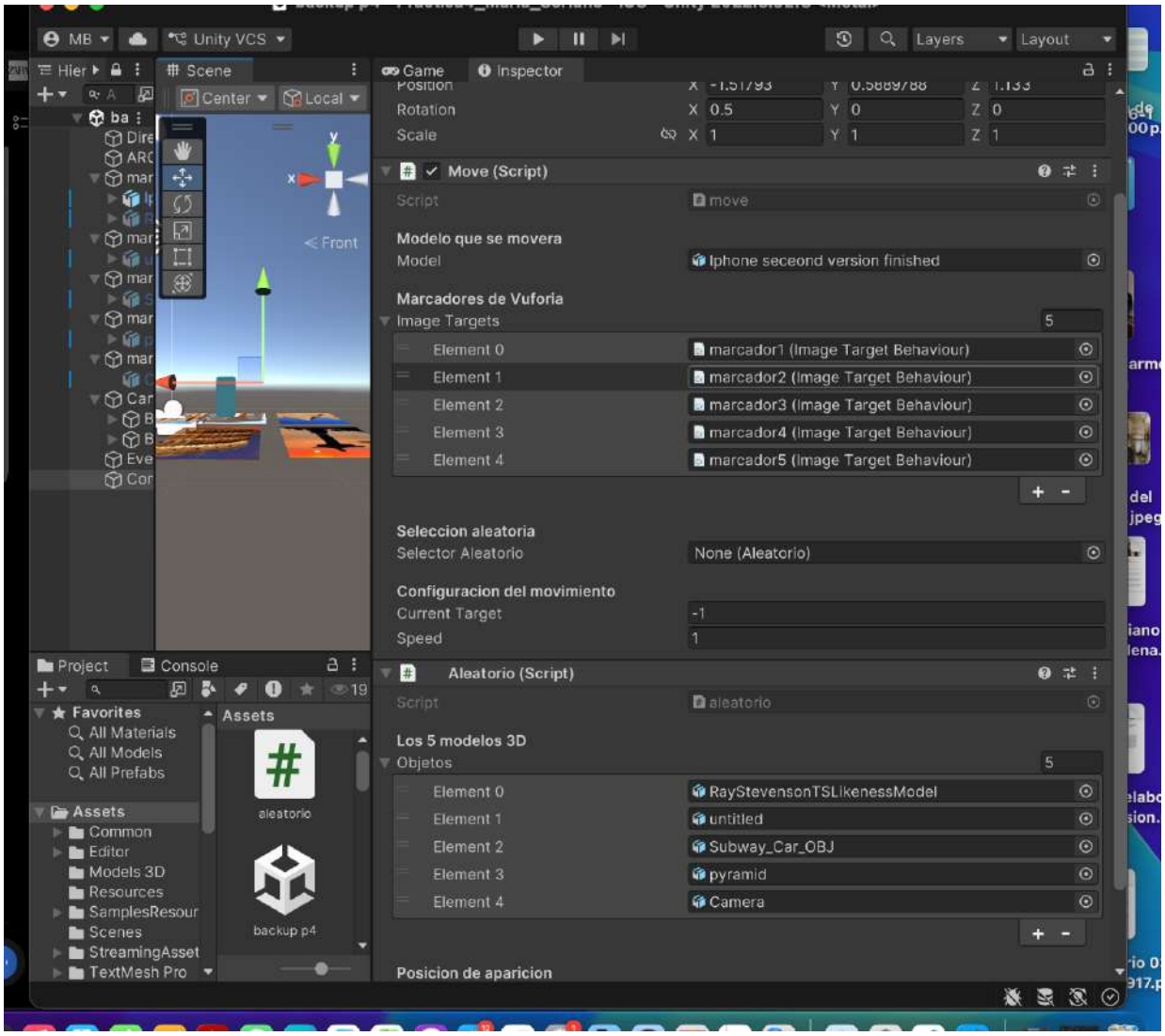


Figura 4. Configuración del movimiento y asignaciones

La captura documenta el componente move y sus referencias: GameObject del iPhone y conjunto de cinco Image Targets. En versiones tempranas de configuración todavía faltaba completar parte de la selección aleatoria; se trata de evidencia de construcción, no necesariamente del estado final.

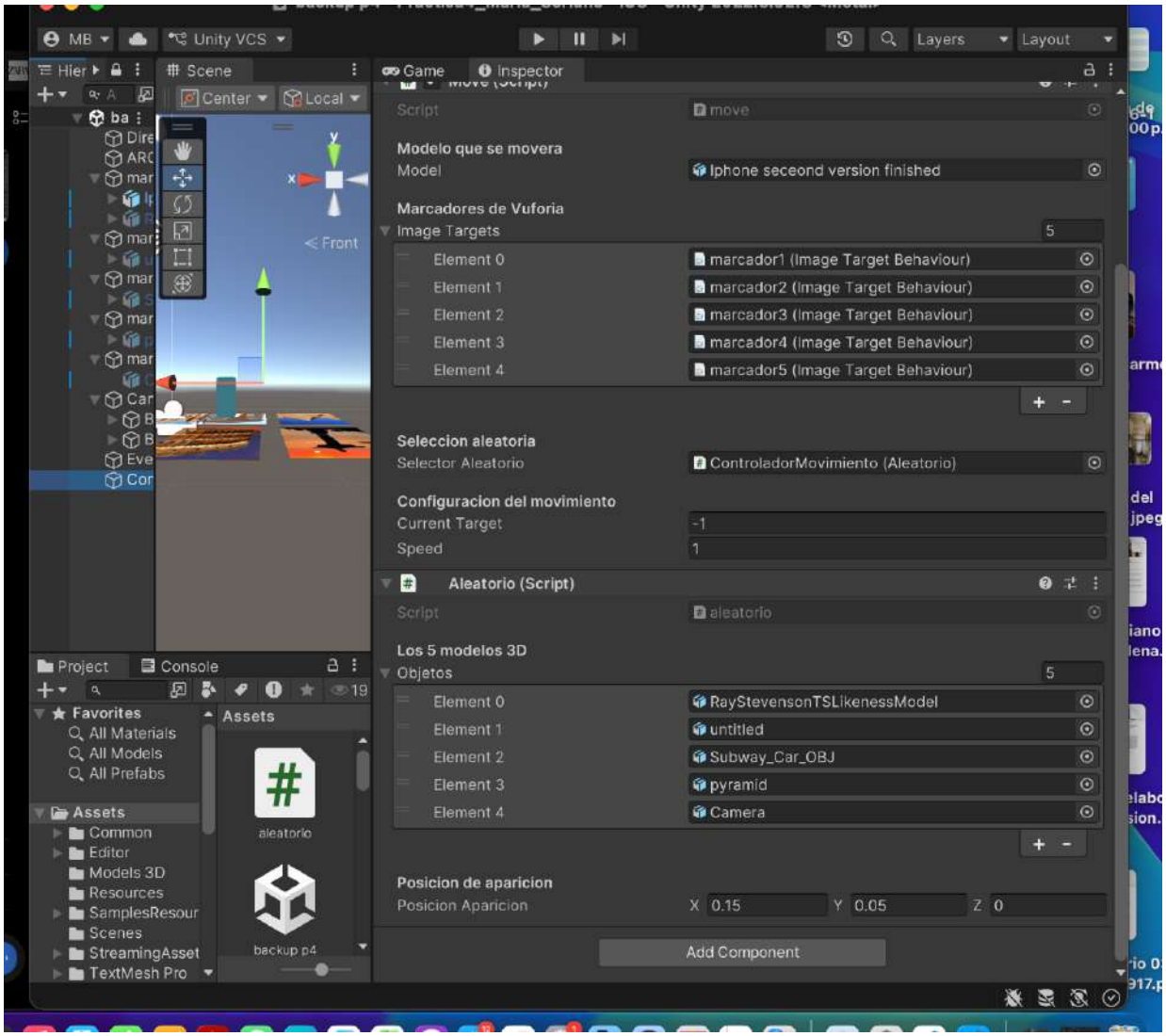


Figura 5. Conexión entre movimiento y selección aleatoria

En Inspector se aprecia la referencia al componente aleatorio del mismo ControladorMovimiento. El arreglo Objetos contiene cinco instancias originales de la escena. Es importante que sean referencias a GameObjects de Hierarchy y que el iPhone no se agregue a ese arreglo, pues funciona como protagonista.

2. Preparación de modelos, materiales y jerarquía

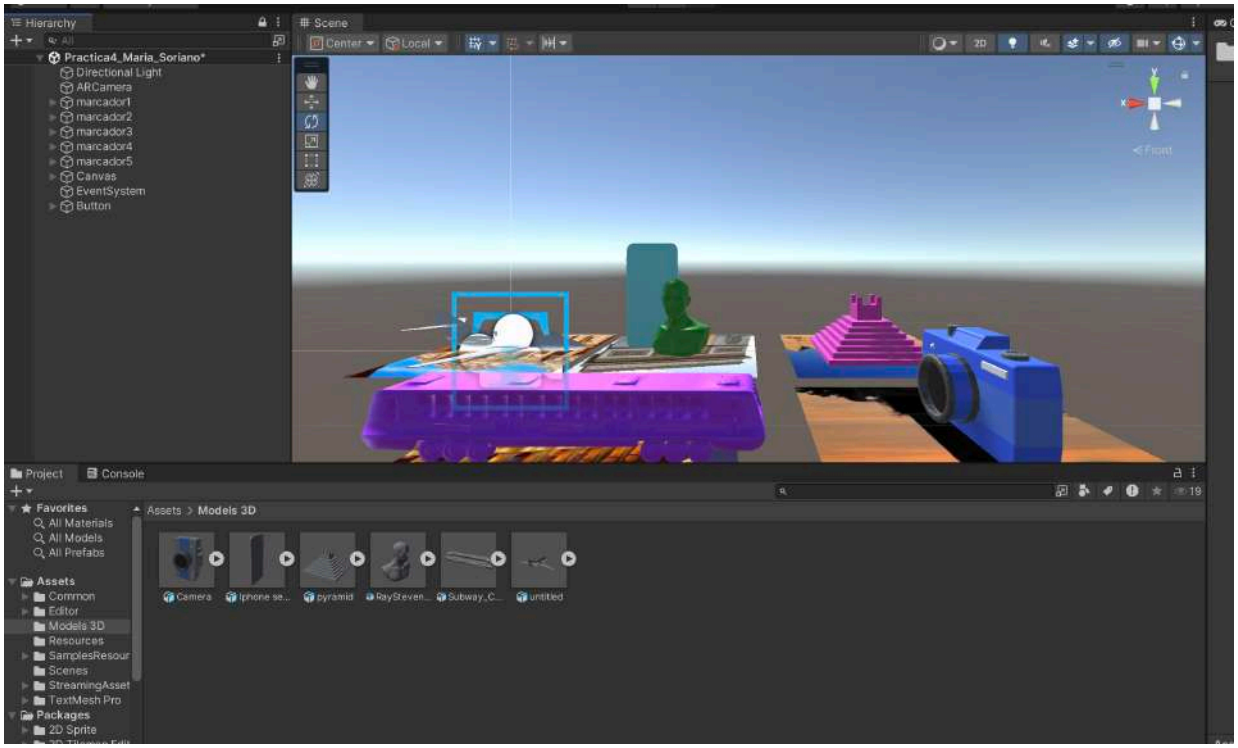


Figura 6. Primera vista de los recursos 3D

En Scene se muestran simultáneamente modelos como el teléfono, busto, pirámide y cámara; la captura permitió detectar problemas de escala y de objetos encimados durante la preparación.

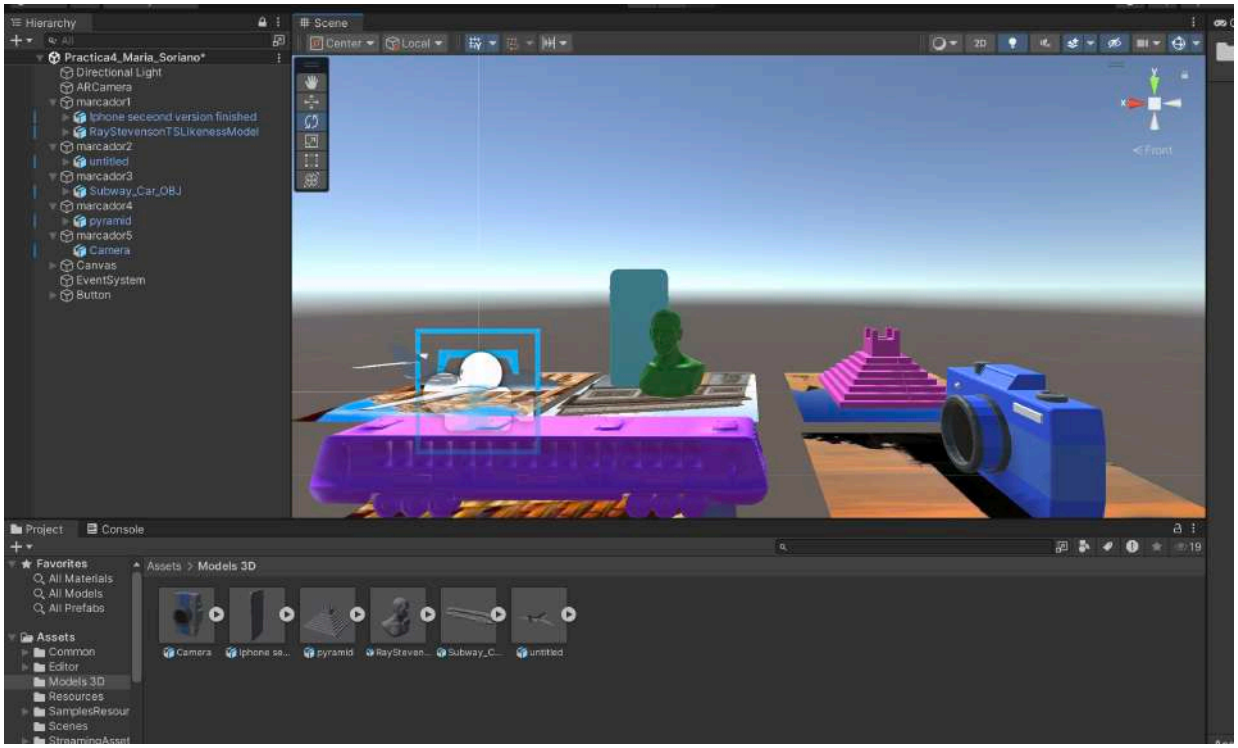


Figura 7. Distribución inicial de objetos

Se inspeccionan las posiciones relativas de los modelos sobre los planos asociados a los marcadores. La vista todavía es de edición y no confirma el comportamiento durante seguimiento real.

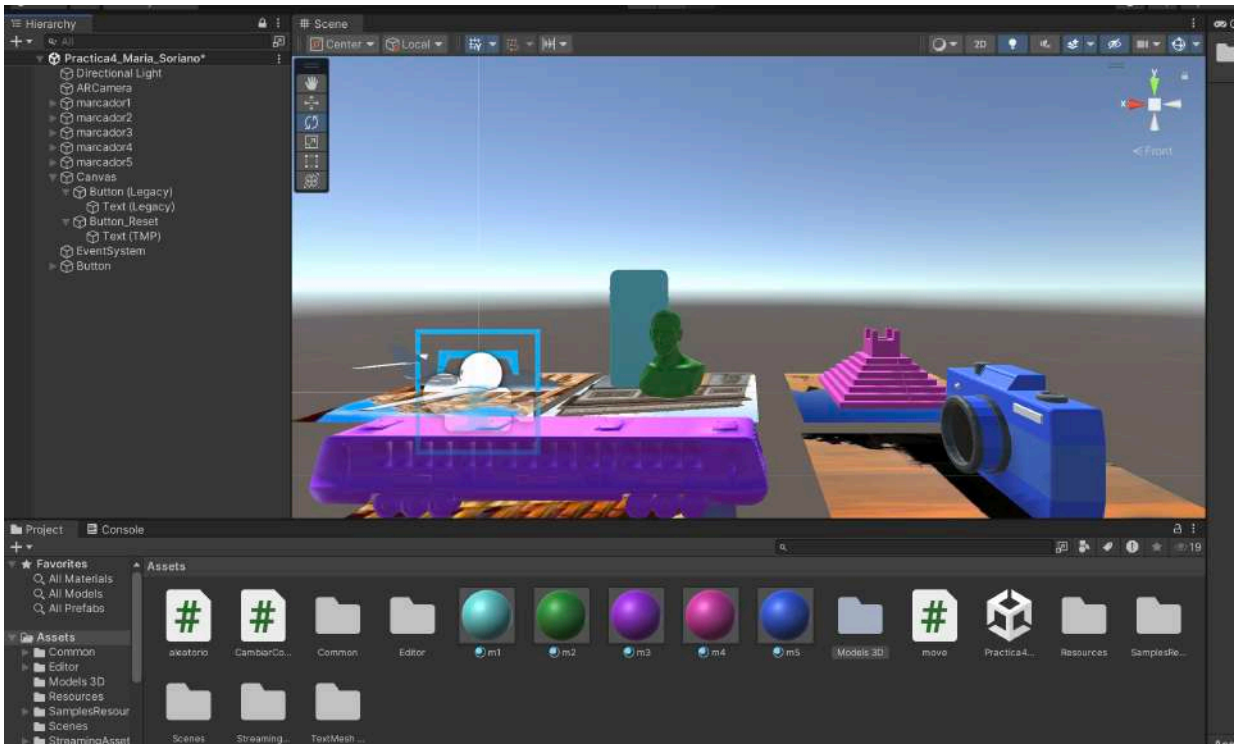


Figura 8. Ubicación de modelos y carpeta de materiales

La segunda secuencia expone los modelos y materiales disponibles en Project, necesarios para configurar colores de representación y distinguir visualmente las alternativas.

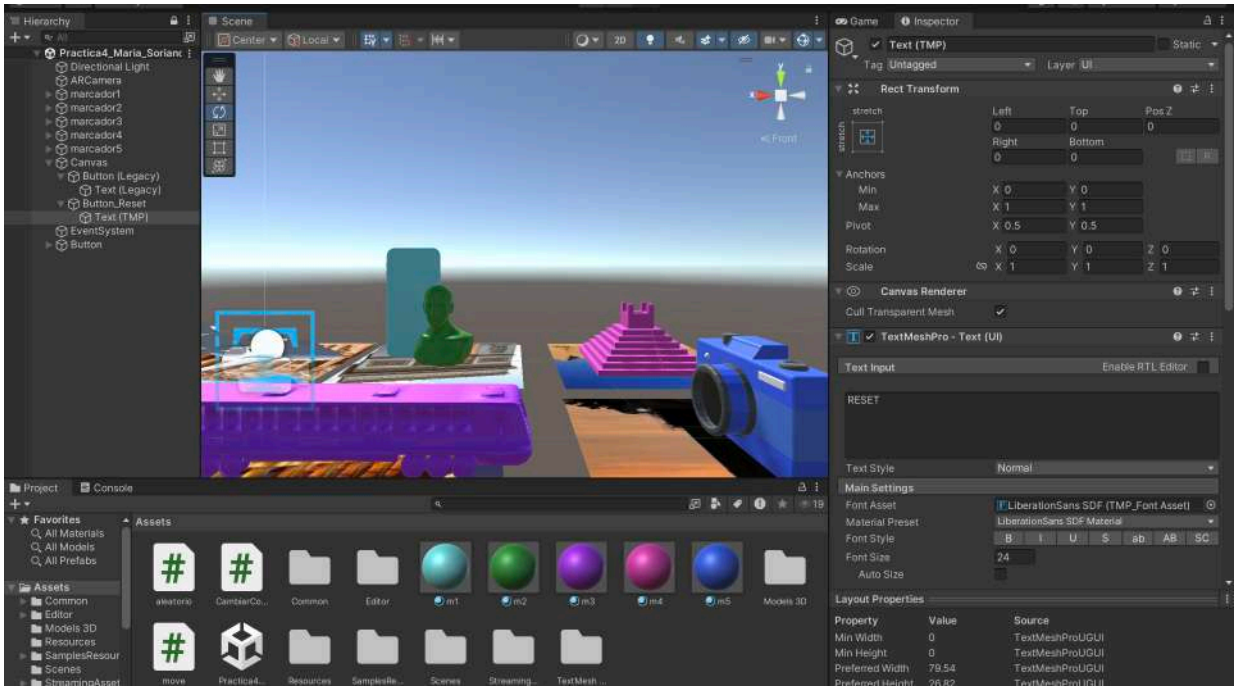


Figura 9. Creación de controles en Canvas

En Hierarchy se ubica Canvas con los objetos de interfaz. Se incorporaron botones para MOVE y RESET; esta captura describe la estructura inicial antes de enlazar las acciones.

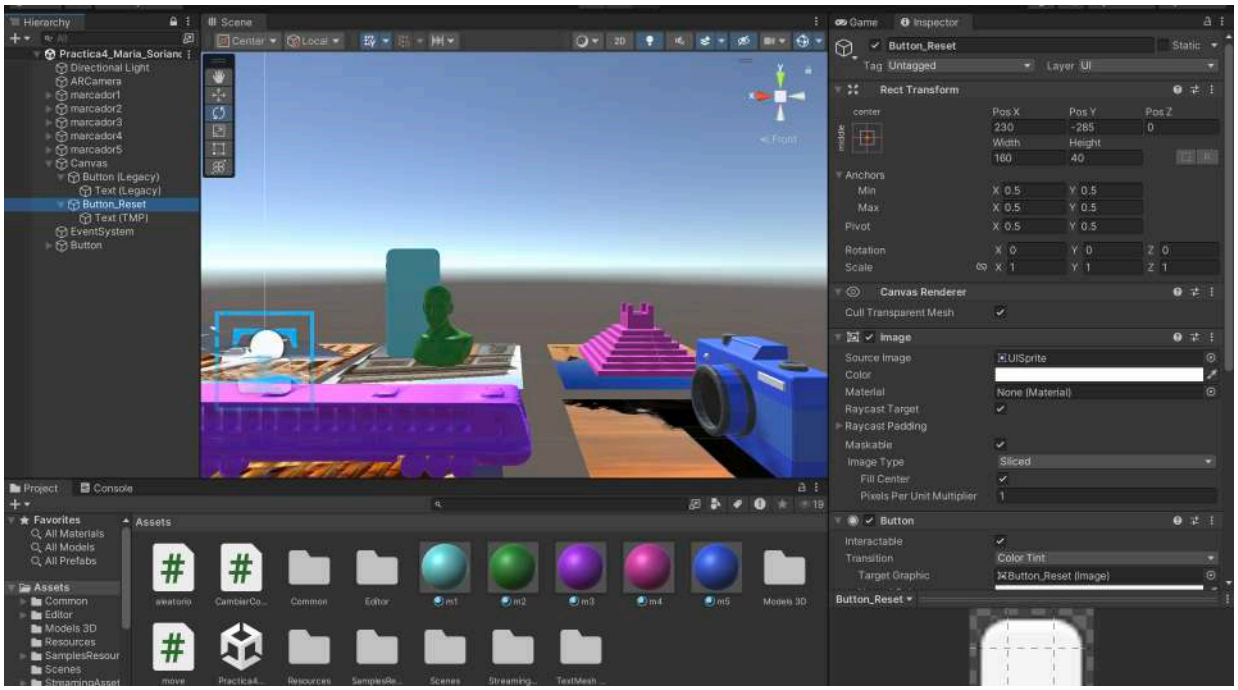


Figura 10. Texto del botón RESET

El objeto Text permite identificar visualmente el botón de reinicio. El texto corresponde al elemento gráfico; su comportamiento real depende del evento On Click del componente Button.

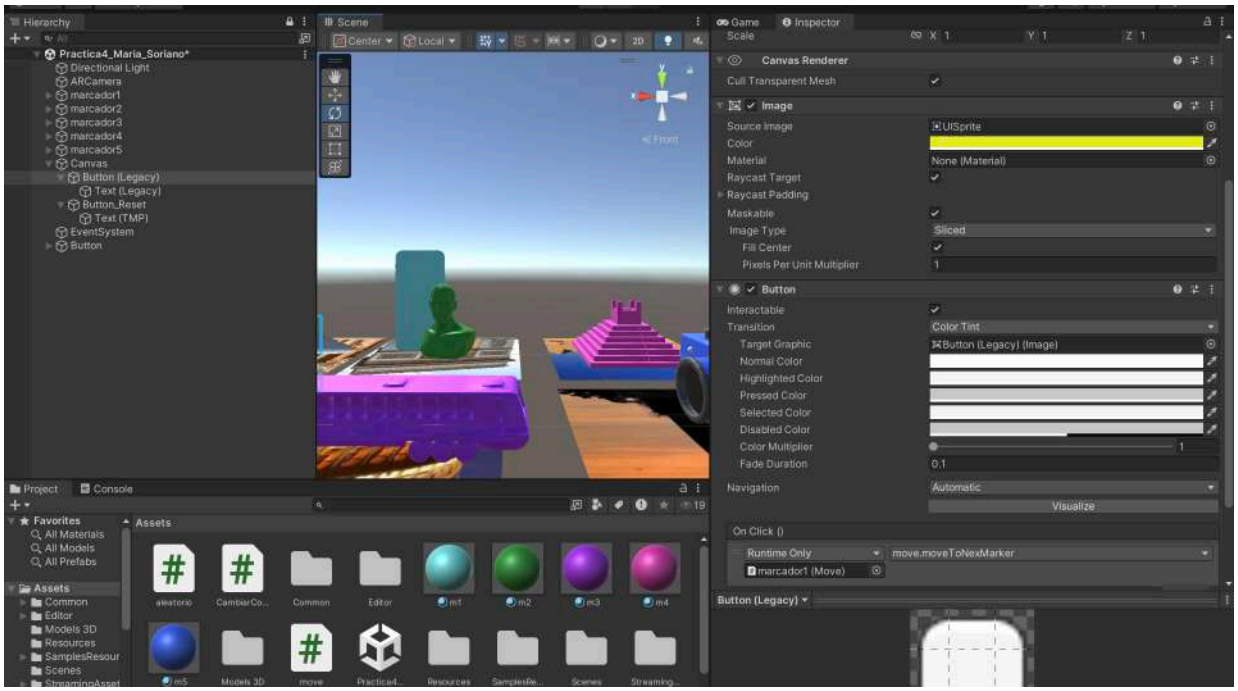


Figura 11. Personalización de botón y color

En Inspector se ajustan las propiedades de Image y el color del elemento de la interfaz. La selección de colores diferencia la acción de mover y la de restablecer.

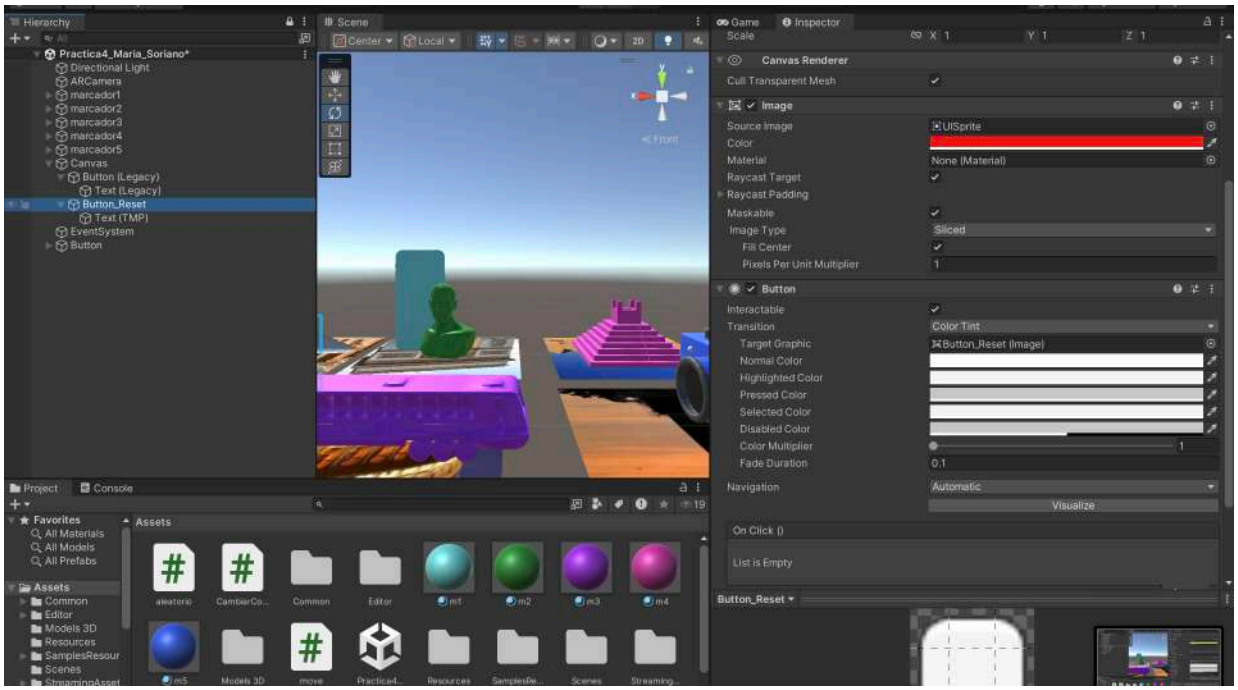


Figura 12. Enlace del botón MOVE

El evento On Click() del botón MOVE apunta a ControladorMovimiento y al método público moveToNexMarker(). La vinculación es obligatoria para que pulsar el botón dispare la búsqueda de otro marcador visible.

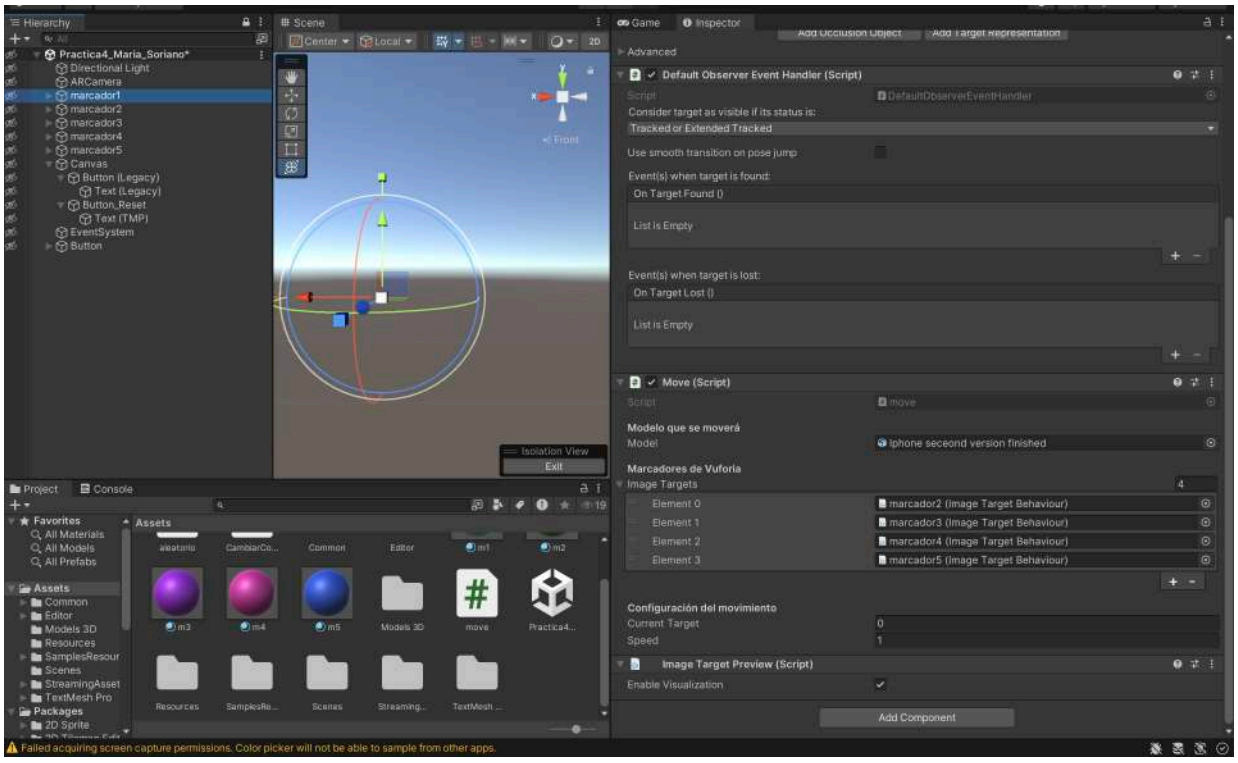


Figura 13. Configuración visual del botón RESET

Se documentan los ajustes del botón de reinicio y su componente Button, incluido el evento de clic. La finalidad es ofrecer una forma de repetir la experiencia sin reiniciar Play Mode.

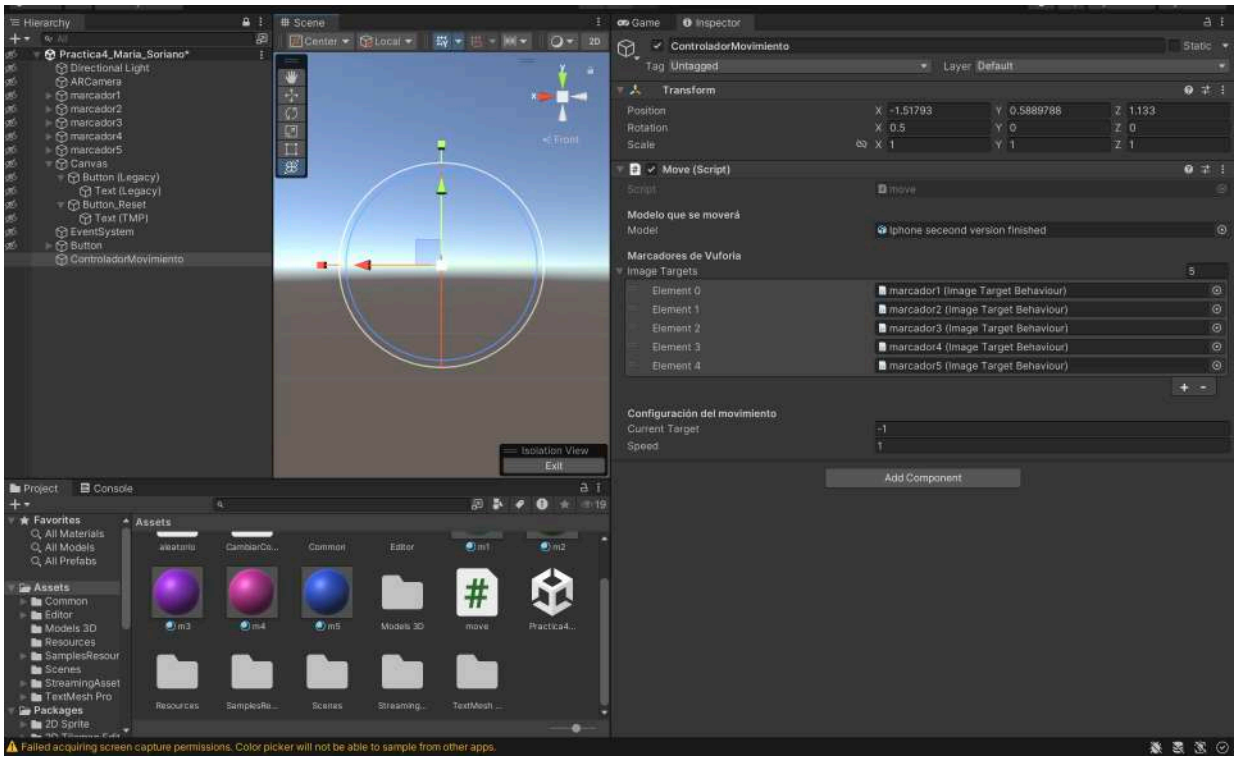


Figura 14. Estructura del Image Target y componentes

Inspector muestra la configuración del marcador seleccionado, incluida su vinculación con Vuforia y los objetos 3D asociados. Los modelos son hijos de los marcadores para permanecer alineados con la imagen reconocida.

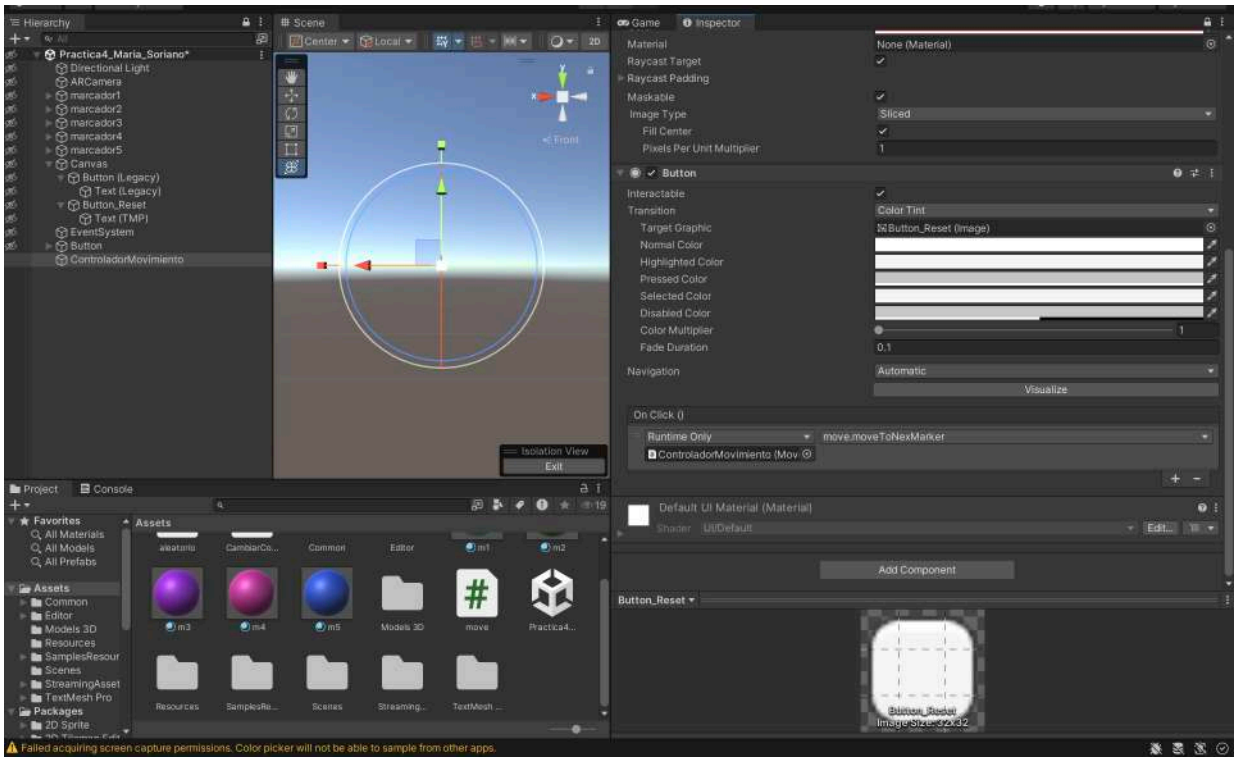


Figura 15. Asociación de Image Targets y materiales

La captura apoya la verificación de que el seguimiento y la jerarquía están vinculados a recursos concretos de la escena. Los materiales visibles tienen función estética y no sustituyen las referencias ObserverBehaviour.

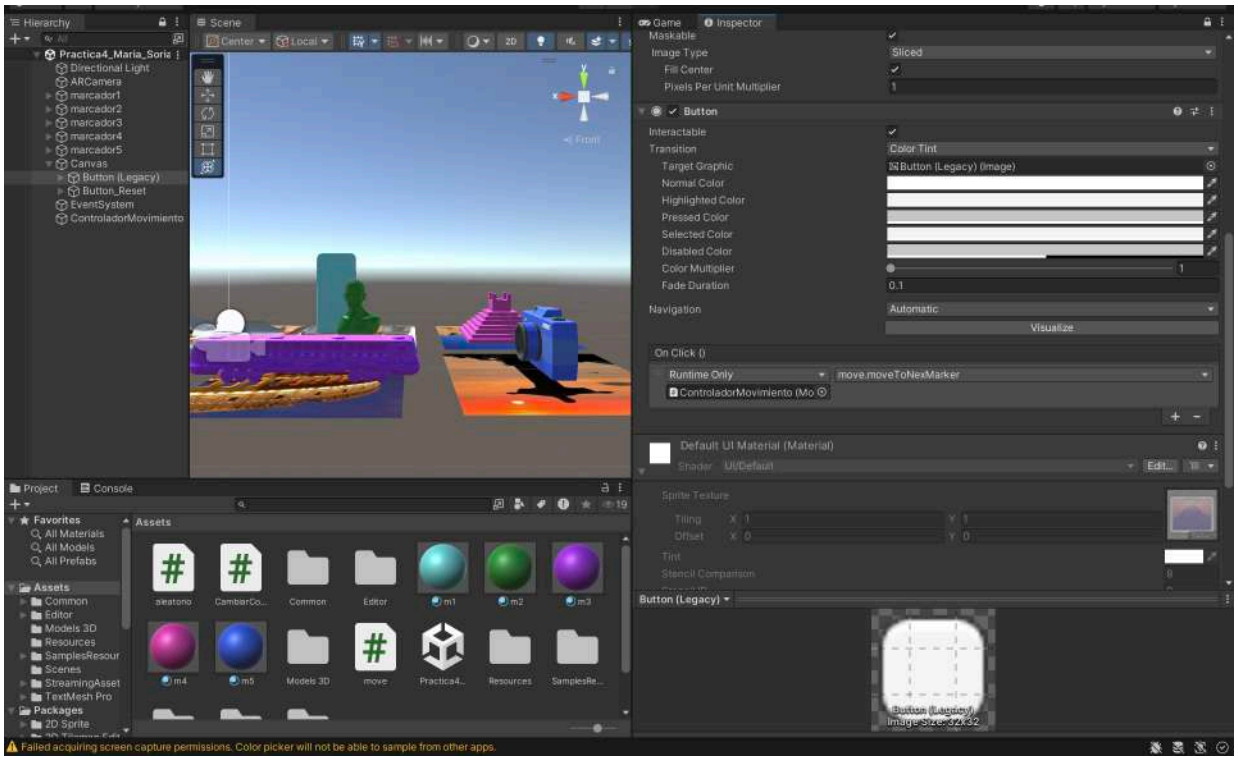


Figura 16. Asignación del modelo principal

Se aprecia ControladorMovimiento con el campo Model enlazado al iPhone y los cinco Image Targets. El controlador guarda la transformación original del iPhone para usarla al reiniciar.

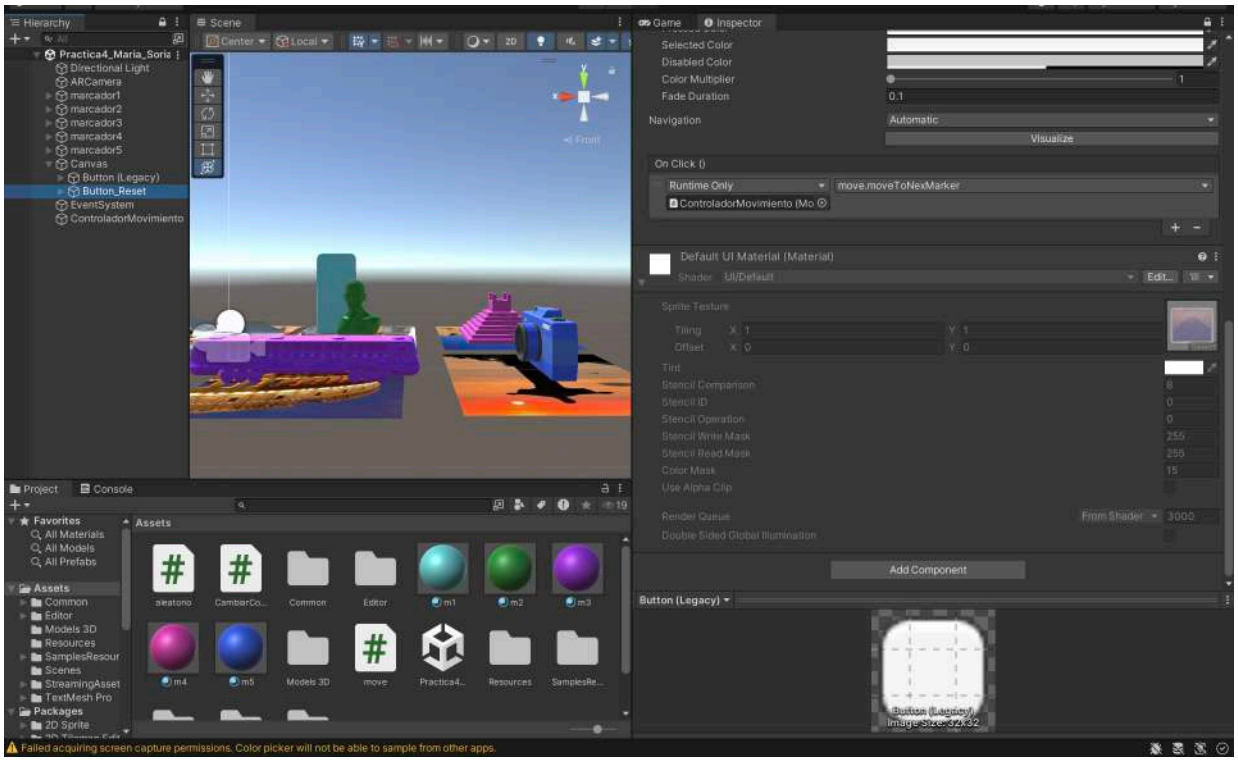


Figura 17. Botón RESET y vínculo del método

El Inspector del botón muestra el evento On Click() que ejecuta Resetear() en move.cs. Este método cancela movimientos en curso y devuelve la transformación inicial.

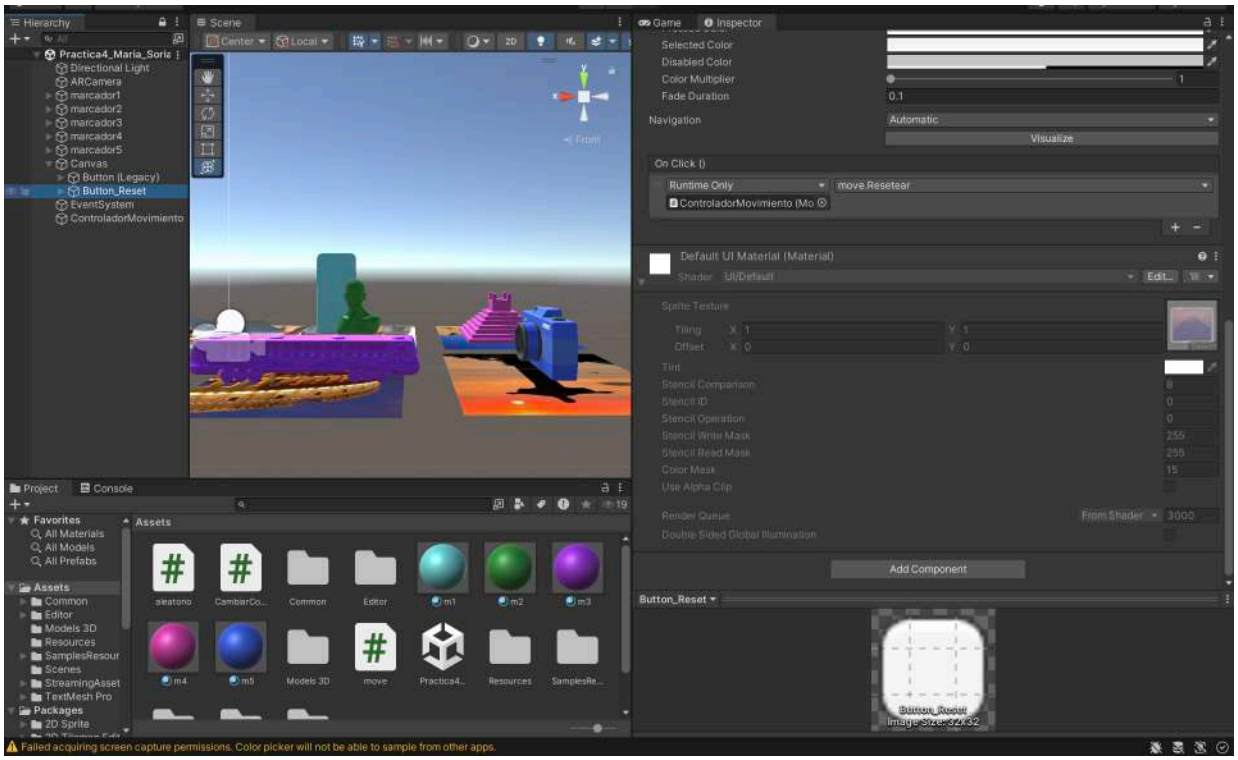


Figura 18. Verificación del elemento UI

La captura presenta el botón seleccionado y su enlace con el controlador. Es una evidencia de configuración de interfaz, no una captura de reproducción.

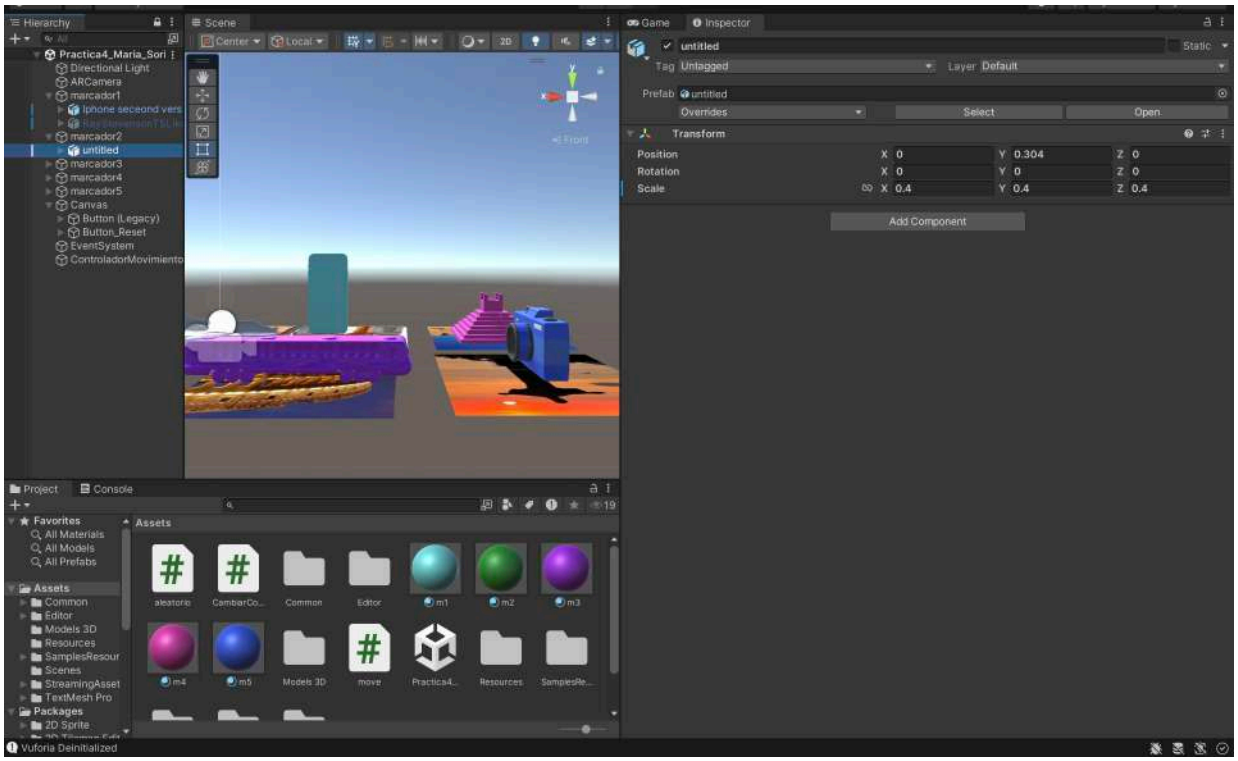


Figura 19. Vista de objeto de origen

Se selecciona un modelo 3D dentro de la jerarquía y se inspeccionan posición y escala. Ajustarlos afecta la visualización de la instancia resultante.

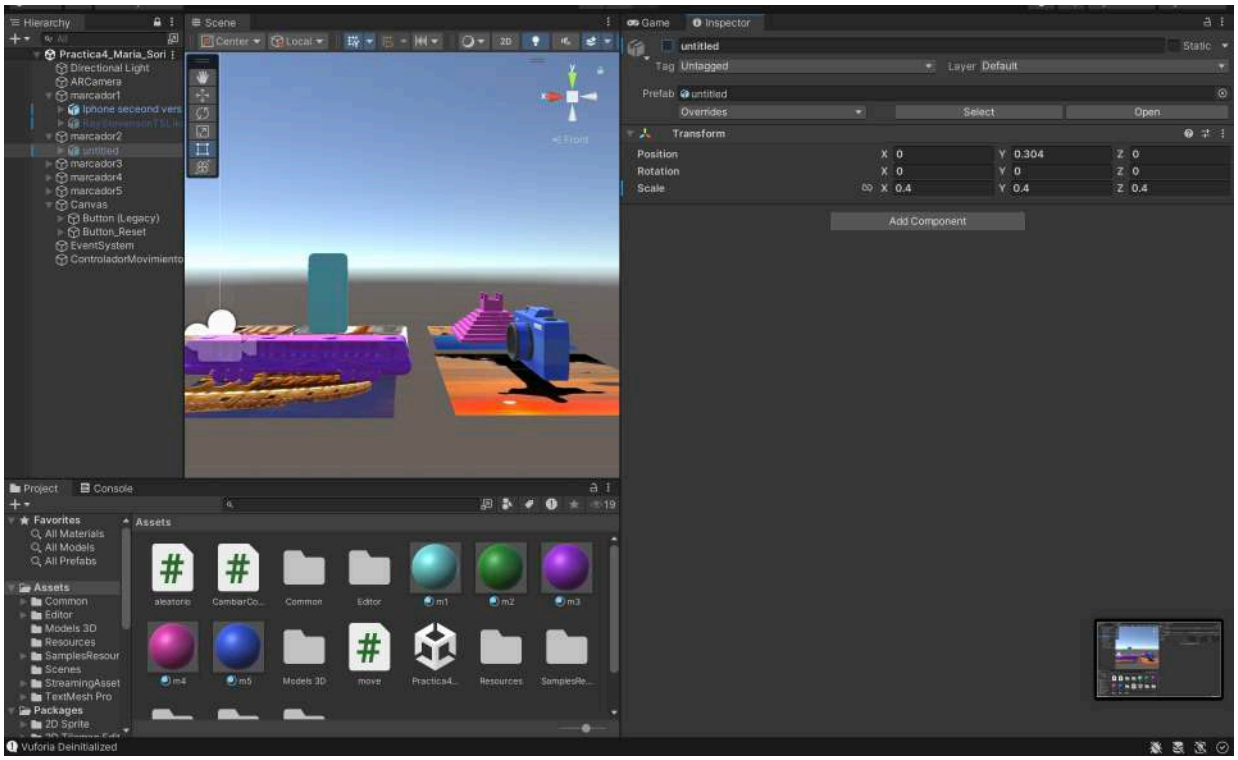


Figura 20. Transform del modelo alternativo

El modelo tiene parámetros locales para colocarlo sobre el marcador. La escala local merece especial atención porque los objetos originales no tienen dimensiones homogéneas.

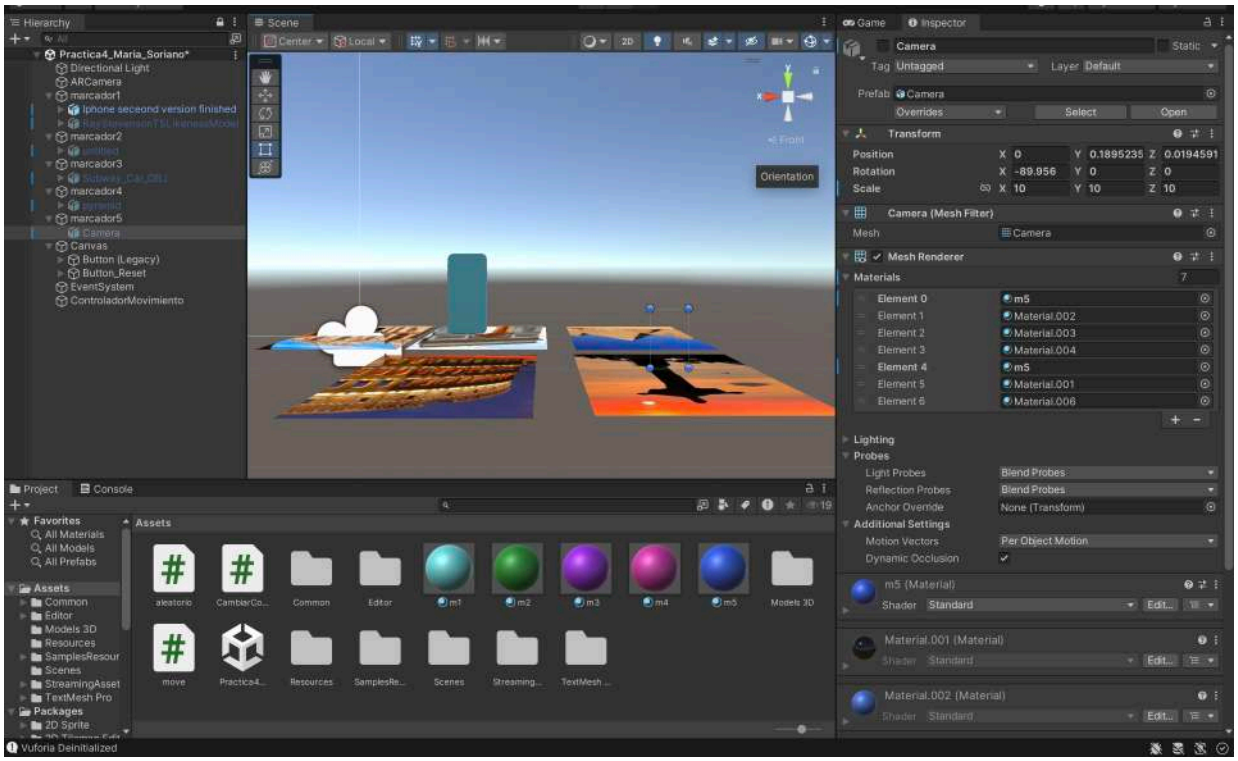


Figura 21. Inspección de cámara y modelo

Los componentes de cámara o Mesh Renderer muestran la geometría visible y sus materiales, relevantes para una presentación reconocible de cada sorpresa.

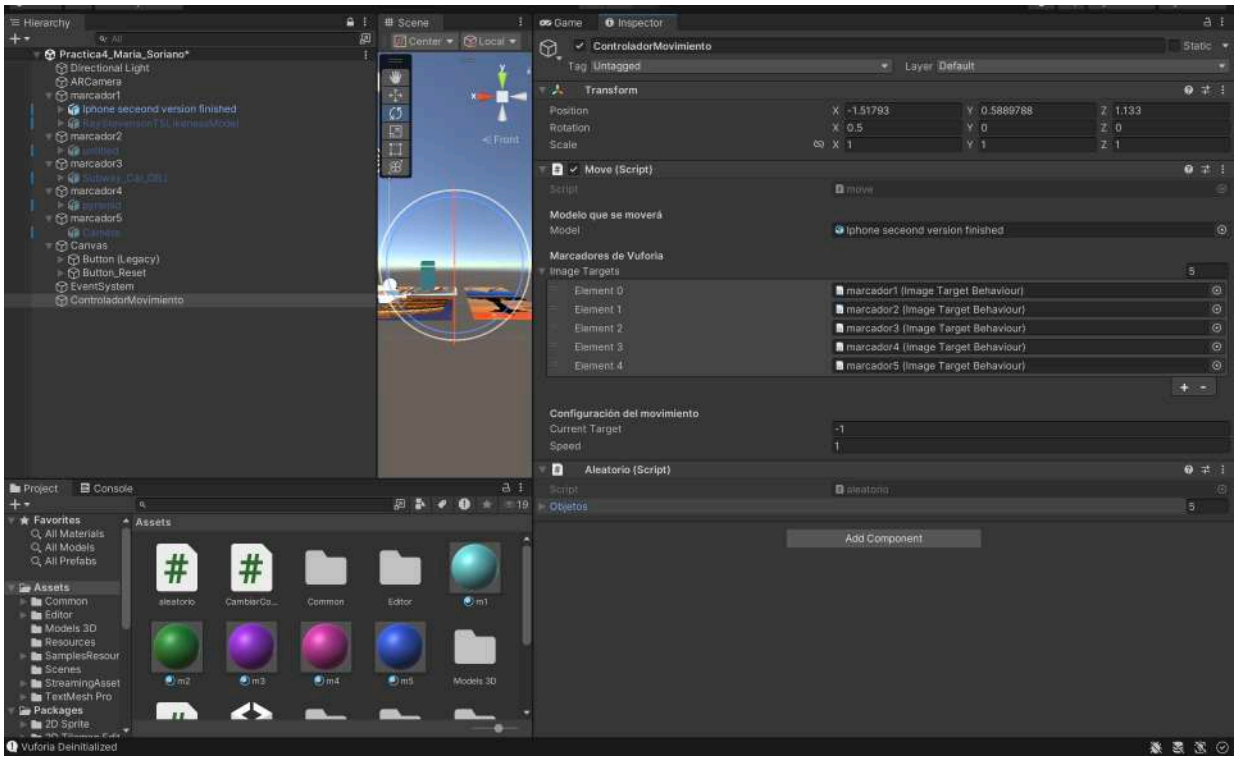


Figura 22. Primera asociación de scripts

En el Inspector se ven move.cs y aleatorio.cs sobre ControladorMovimiento. La selección del modelo ocurre solo después de alcanzar un marcador detectado.

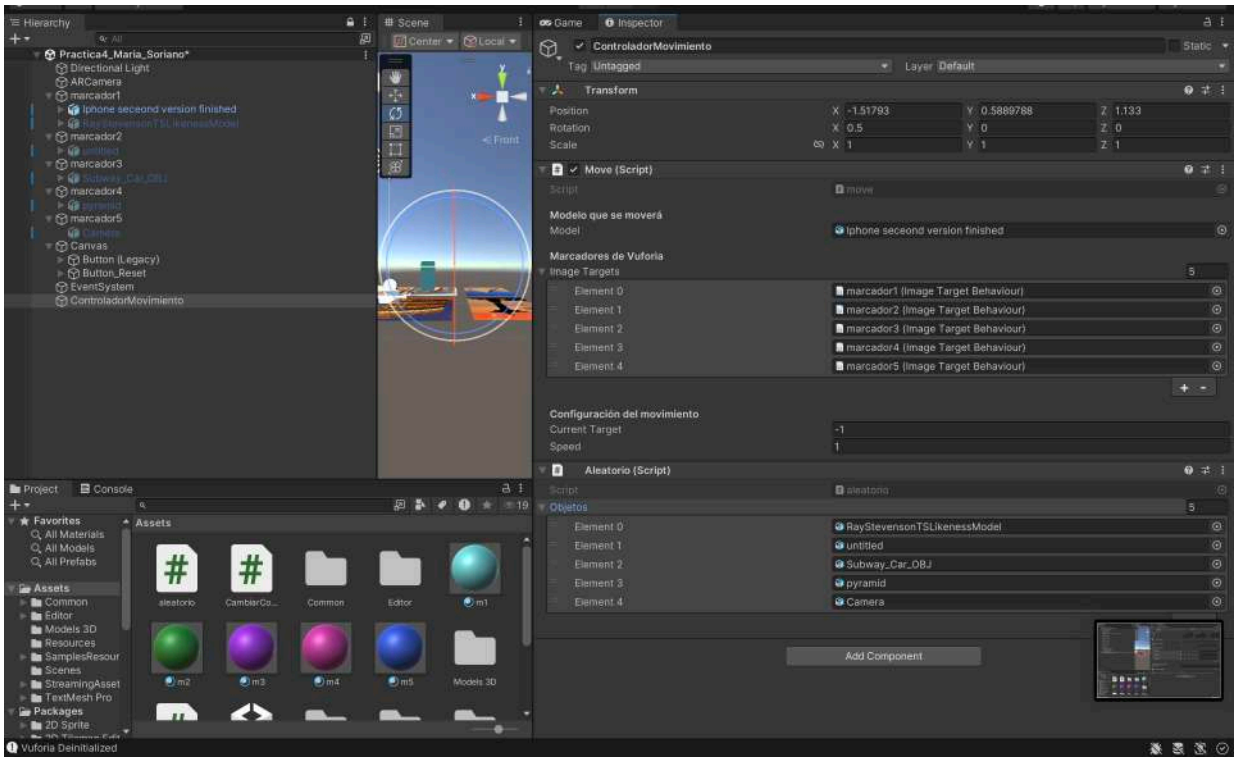


Figura 23. Arreglo de cinco objetos

Se asignan busto, avión, tren, pirámide y cámara a Element 0–4. El orden es significativo porque el switch ObtenerMision() usa el índice para elegir la leyenda correcta.

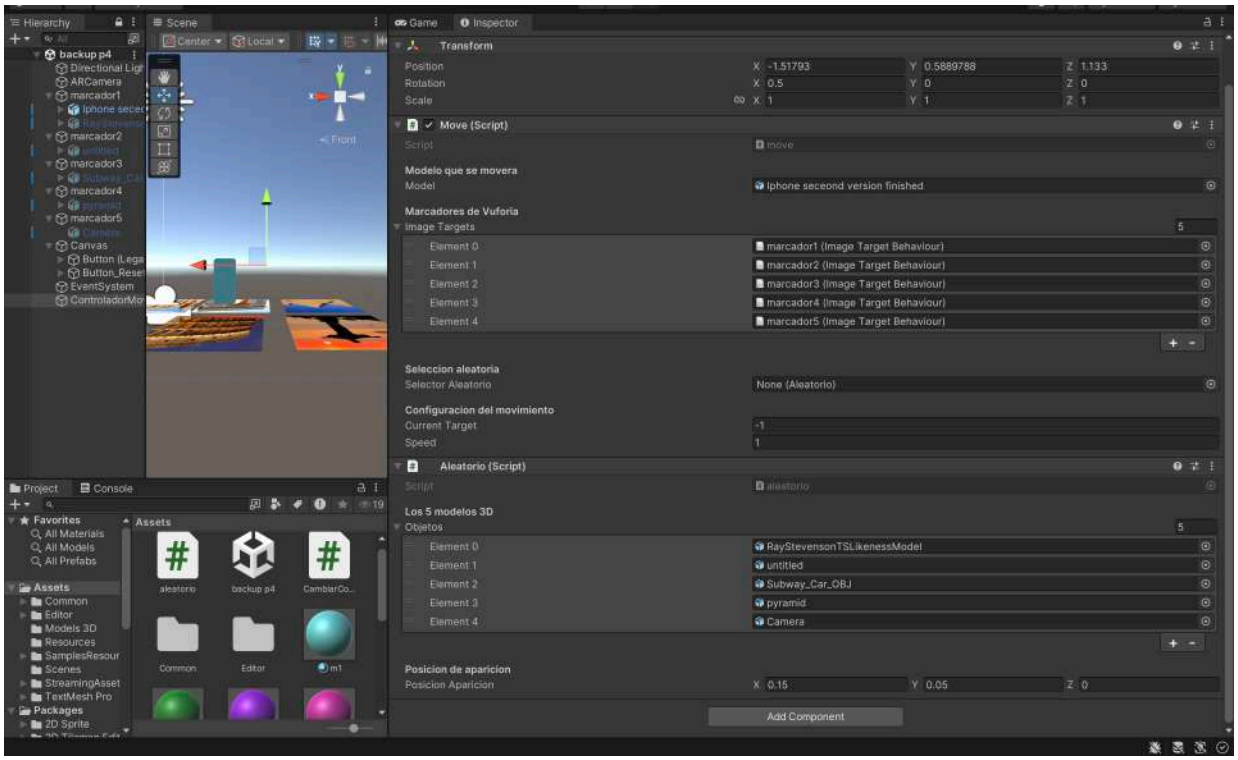


Figura 24. Referencia cruzada al selector

El campo Selector Aleatorio vincula el componente move al componente aleatorio. Si la referencia está vacía, el código intenta obtenerlo desde el mismo objeto usando GetComponent.

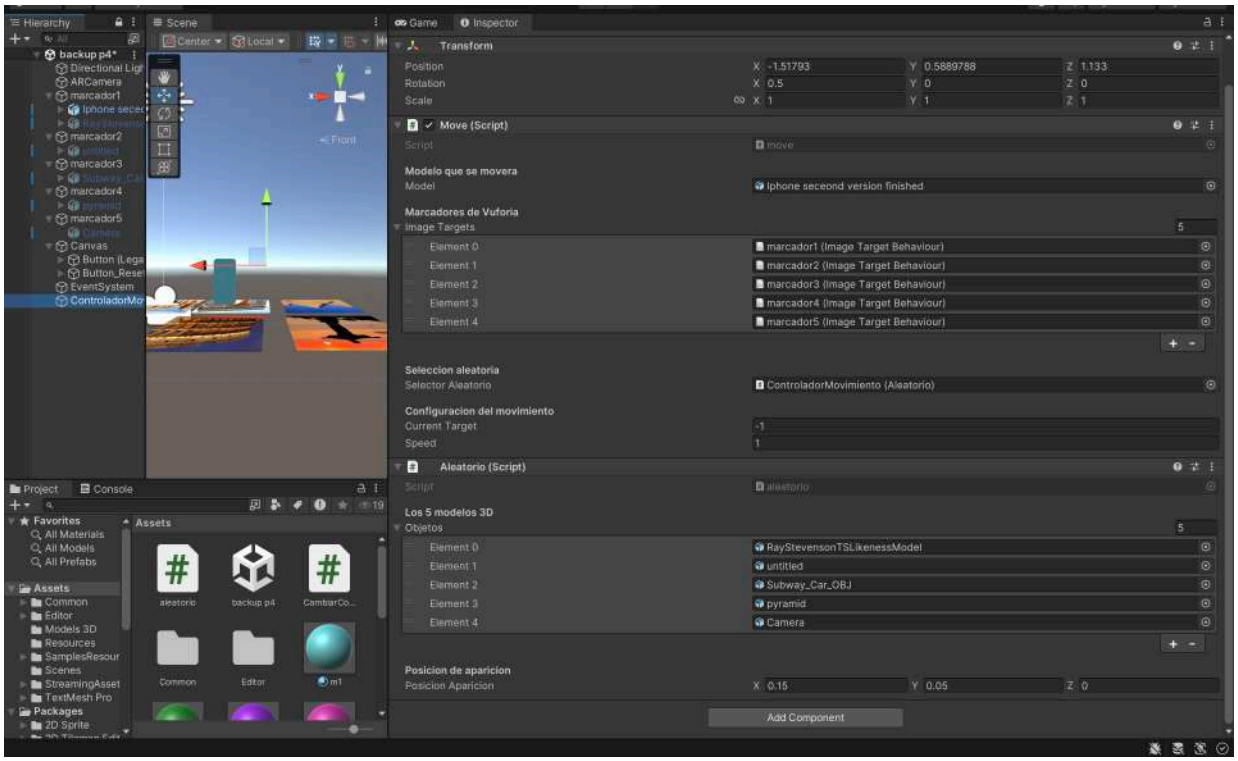


Figura 25. Activación y actualización del controlador

Tras editar el script, Inspector vuelve a mostrar los campos. Unity conserva las referencias serializadas mientras los nombres y tipos públicos permanezcan compatibles.

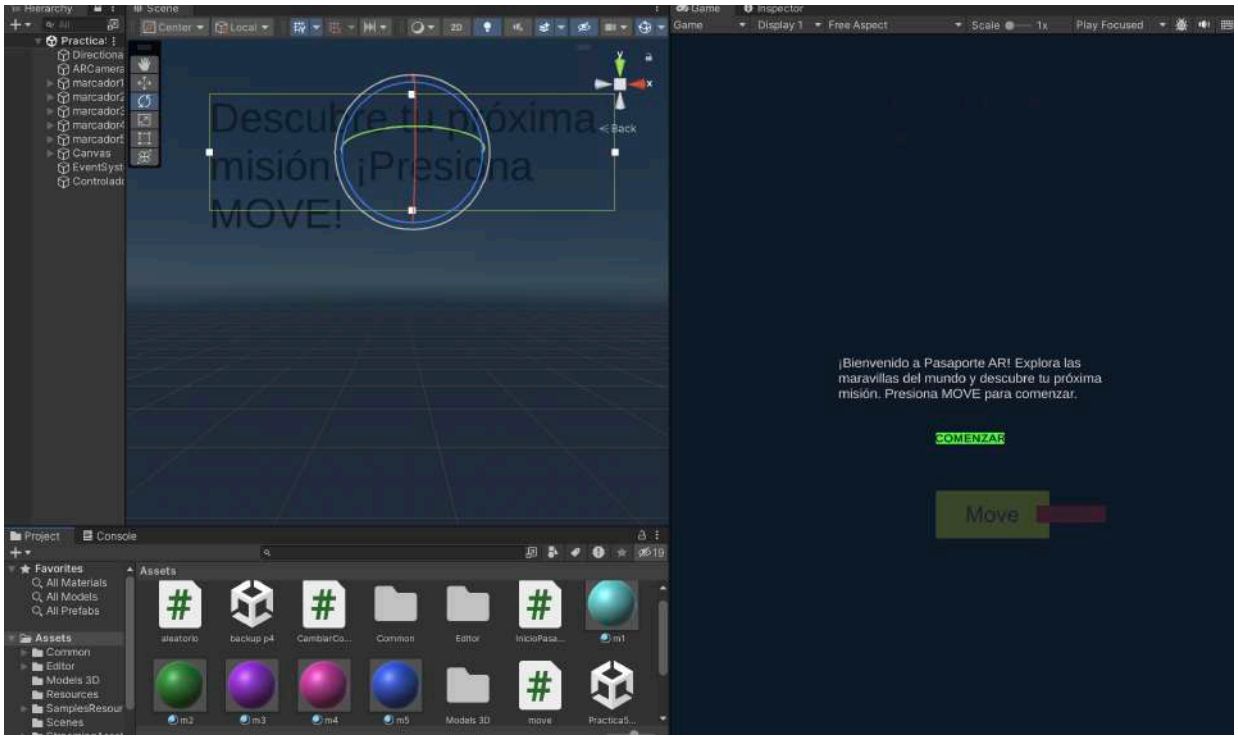


Figura 26. Construcción de la interfaz de bienvenida

La interfaz incluye un mensaje introductorio y el botón COMENZAR. La función de este diseño es explicar el propósito antes de mostrar controles y cámara.

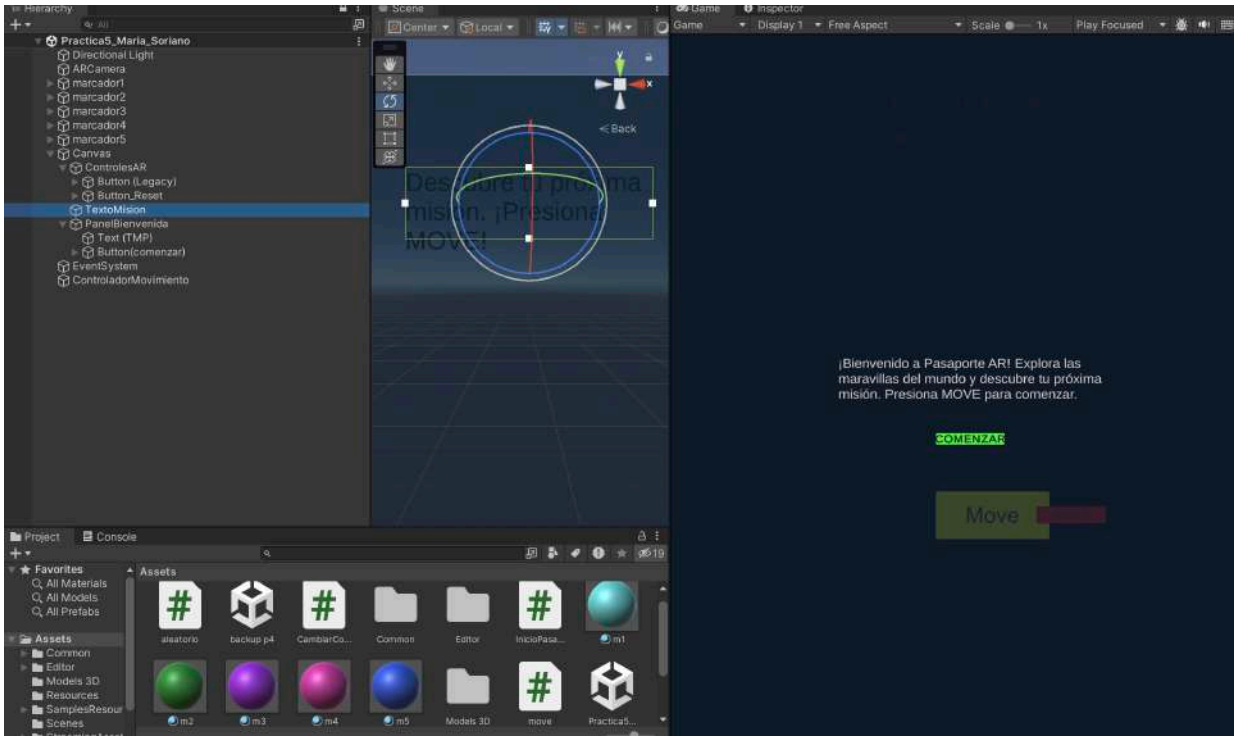


Figura 27. Vista previa de la pantalla de inicio

La captura de Game ilustra cómo la bienvenida se presenta delante del contenido AR. Permite detectar tamaño, contraste y alineación de textos.

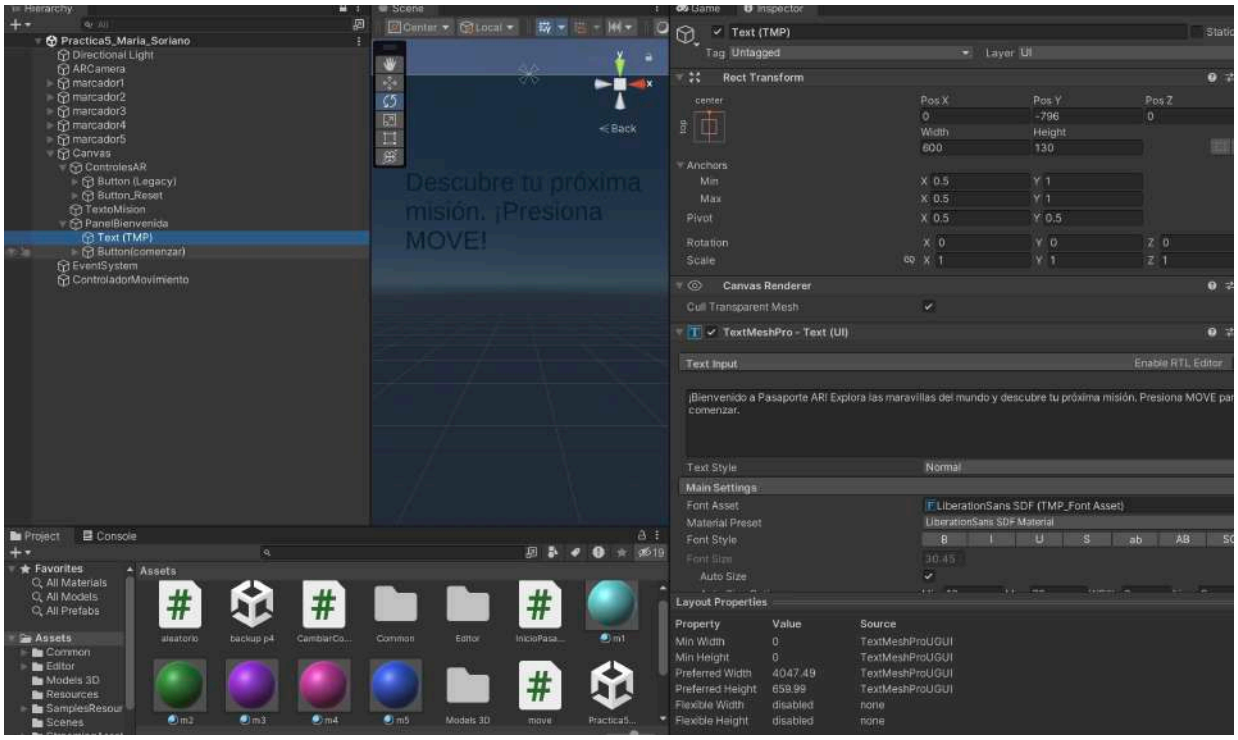


Figura 28. Creación y ajuste de TextoMision

Se incorpora el componente TextMeshPro para mostrar mensajes cambiantes. Debe estar asignado a aleatorio.textoMision para recibir las leyendas que retorna ObtenerMision().

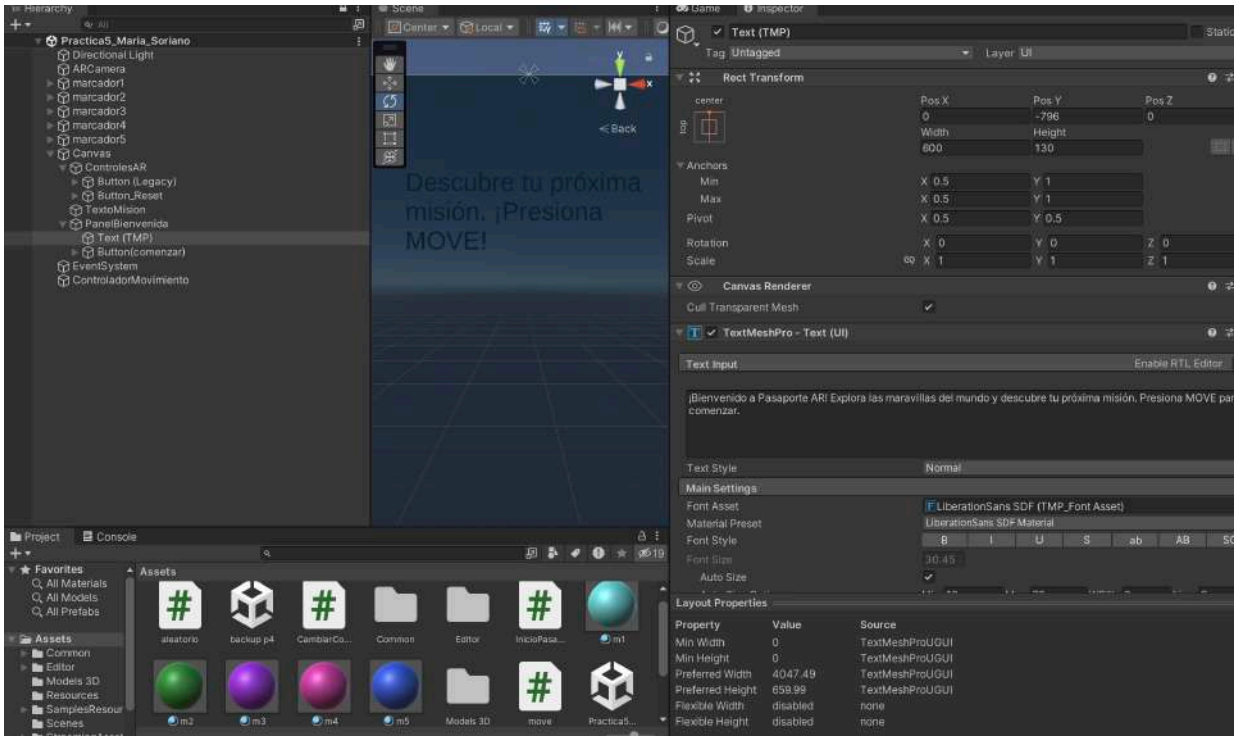


Figura 29. Edición de tamaños y anclajes

Los campos de Rect Transform y el área de texto determinan si la misión cabe en la pantalla. Los anclajes superiores permiten mantener el texto próximo a la parte alta del dispositivo.

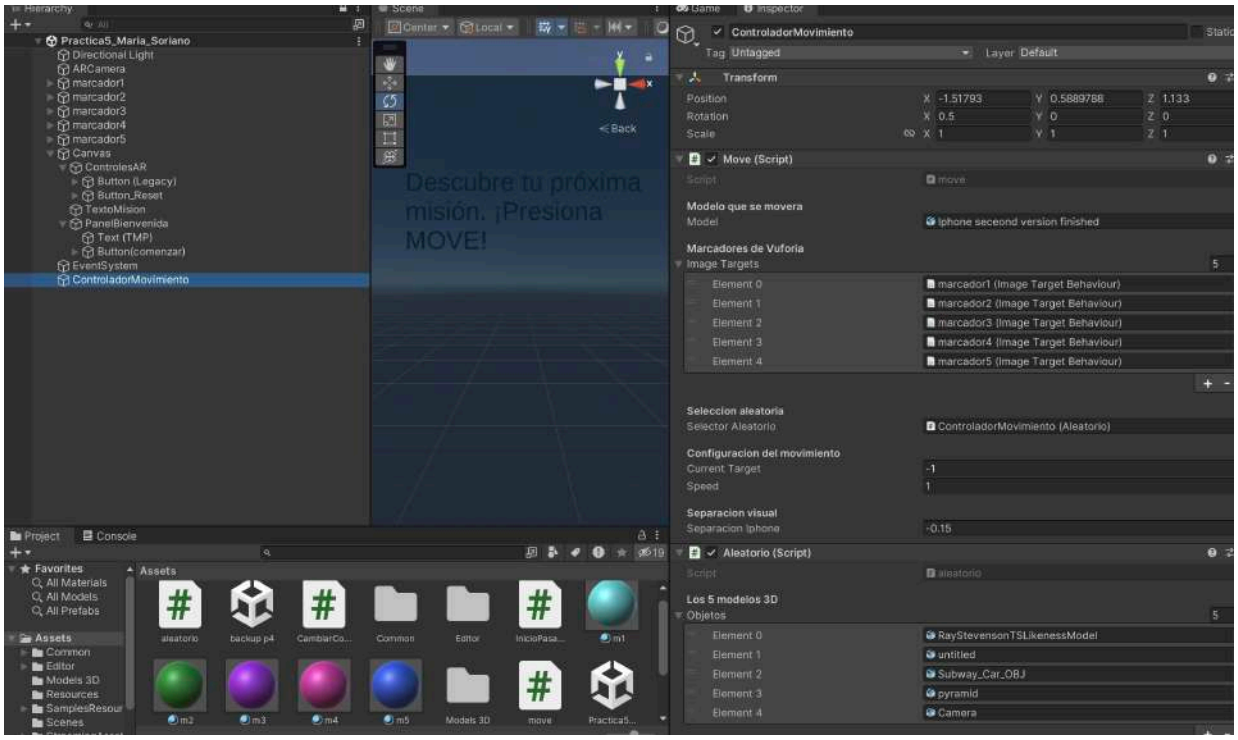


Figura 30. Validación de tipografía de la misión

Las capturas muestran ajustes de TextMeshPro (tipo, tamaño y contenedor). Con fondos de cámara variables, conviene usar texto claro y un fondo oscuro semitransparente para mejorar legibilidad.

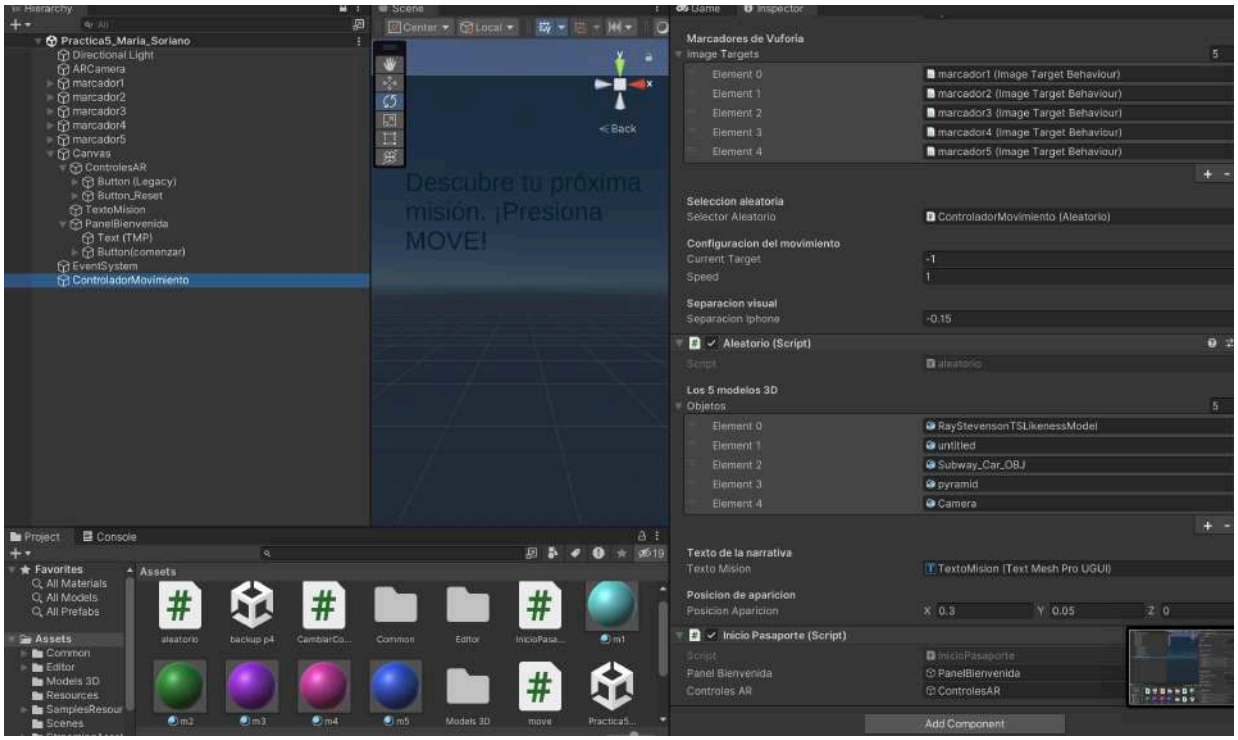


Figura 31. Panel de bienvenida y controles

Se distingue la jerarquía de Canvas, PanelBienvenida y ControlesAR. InicioPasaporte conmuta su visibilidad al pulsar COMENZAR, sin modificar los modelos de la escena.

3. Lógica de la narrativa y comportamiento esperado

Pasaporte AR presenta un recorrido por imágenes de lugares emblemáticos: el usuario inicia en una pantalla de bienvenida, pulsa COMENZAR y, cuando Vuforia identifica otro marcador, activa MOVE. El iPhone se desplaza hacia ese destino; al llegar se crea una copia aleatoria de uno de cinco modelos y se escribe una misión contextual. RESET devuelve el recorrido al origen.

Índice / modelo	Misión programada
0 · Busto	Visita un museo: descubre una obra de arte o un personaje histórico.
1 · Avión	Viaja en avión: prepárate para descubrir el siguiente destino.
2 · Tren	Explora en tren: recorre la ciudad y conoce un lugar histórico.
3 · Pirámide	Descubre un monumento: investiga un dato sobre las pirámides.
4 · Cámara	Captura un recuerdo: toma una fotografía del monumento visitado.

Estas misiones son textos de la interfaz; el código documentado no comprueba que la persona realice físicamente la actividad ni valida respuestas. Su propósito es introducir la narrativa y relacionarla con el modelo generado.

4. Explicación funcional de los tres scripts

4.1 aleatorio.cs — selección, aparición y texto

Objetos, textoMision y posicionAparicion — objetos referencia los cinco originales. textoMision es una referencia TMP_Text que permite editar en tiempo real el contenido del Canvas. posicionAparicion establece el desplazamiento local de la sorpresa respecto al Image Target de destino.

ultimoIndice y modeloActual — ultimoIndice registra la selección previa para evitar que se repita inmediatamente; modeloActual conserva la instancia temporal y permite eliminarla antes del siguiente sorteo.

Start() — Escribe un mensaje inicial invitando a usar MOVE mediante MostrarMensaje(). No inicia un sorteo por sí mismo.

MostrarAleatorio(Transform destino) — Valida que existan objetos y destino, oculta la instancia anterior, elige índice con Random.Range dentro de un do-while que descarta repetir la selección previa; luego instancia una copia como hija del marcador recibido.

Instantiate(original, destino, false) — Crea un nuevo GameObject asociado al marcador de destino. La posición, rotación y escala se asignan después usando transform.localPosition, localRotation y localScale; se activa la instancia con SetActive(true).

ObtenerMision(int indice) — Implementa un switch con cinco textos. La relación depende del orden del arreglo Objetos, por lo que cambiar ese orden sin ajustar el switch mostraría una misión incorrecta.

MostrarMensaje(string mensaje) — Comprueba que textoMision no sea null y actualiza la propiedad text. Si el campo no está asignado en Inspector, el modelo puede aparecer pero la leyenda no cambiará.

OcultarModelo() — Destruye exclusivamente la última copia creada y borra su referencia; no borra los modelos de origen configurados en Hierarchy.

ResetearAleatorio() — Elimina el último modelo, reinicia el índice anterior a -1 y restablece el mensaje de invitación para poder realizar otro recorrido.

4.2 move.cs — seguimiento y movimiento

model, ImageTargets, selectorAleatorio — Son las tres referencias operativas: iPhone a desplazar, arreglo de componentes ObserverBehaviour y componente que gestiona la sorpresa. Sin ellos no se obtiene el flujo integral.

currentTarget, speed, separacionIphone — currentTarget se usa para iniciar la búsqueda circular; speed controla la duración interpolada (1/speed, con protección contra cero); separacionIphone agrega desplazamiento negativo en X para dejar espacio al modelo sorpresa.

padreInicial, posicionInicial, rotacionInicial, escalaInicial — Guardan el estado original del teléfono. Son necesarios cuando se cambia temporalmente el padre con SetParent y al presionar RESET.

Start() — Comprueba la asignación del iPhone, conserva su Transform inicial y, cuando no se enlazó explícitamente, intenta localizar aleatorio en el mismo controlador.

moveToNexMarker() — Evita arrancar dos corrutinas simultáneas con isMoving. Llama a BuscarSiguienteMarcador(), emite aviso si no existe uno disponible y, si lo hay, inicia MoverIphone().

BuscarSiguienteMarcador() — Recorre circularmente ImageTargets desde currentTarget, omite el marcador actual y acepta el primero cuyo estado sea reconocido. Por ello no está obligado a seguir la numeración si una imagen no se detecta.

MarcadorDetectado() — Usa TargetStatus.Status para aceptar TRACKED o EXTENDED_TRACKED. Un marcador configurado pero NOT_OBSERVED no se considera destino válido.

MoverIphone() — Oculta la sorpresa anterior; conserva la posición global del iPhone; lo desacopla temporalmente con SetParent(null,true), calcula la posición de destino mediante TransformPoint y utiliza Lerp con SmoothStep a lo largo de varios fotogramas.

Pérdida de seguimiento — La corrutina verifica cada fotograma si el destino continúa reconocido. Si deja de estarlo, ejecuta RestaurarMarcadorActual() y sale sin dar por finalizado ese desplazamiento.

Llegada al marcador — Al terminar la interpolación, el teléfono se vuelve hijo del Image Target destino, recupera rotación y escala y se ubica con separación en X. Entonces invoca selectorAleatorio.MostrarAleatorio(destino.transform).

RestaurarMarcadorActual() — Regresa el iPhone al último marcador confirmado y restaura sus propiedades locales; cuando vuelve al padre inicial utiliza la posición original sin desplazamiento.

Resetear() — Cancela corrutinas, resetea el selector y reubica el iPhone en marcador1 con su transformación inicial, devolviendo el estado a un nuevo intento.

4.3 InicioPasaporte.cs — pantalla inicial

panelBienvenida y controlesAR — Son las referencias al panel que cubre la pantalla al entrar y al contenedor de botones MOVE/RESET.

Start() — Muestra PanelBienvenida y oculta ControlesAR para impedir iniciar el movimiento antes de leer la introducción.

Comenzar() — Oculta la bienvenida y activa los controles. La función se conecta al evento On Click() del botón COMENZAR.

Los tres componentes trabajan juntos: InicioPasaporte habilita la experiencia; move determina el marcador de llegada; aleatorio crea el elemento elegido y actualiza la misión. Ninguno de los tres scripts implementa por sí mismo cambio de accesorio o de color del personaje; esos criterios deben comprobarse mediante otras funciones o demostraciones.

5. Pruebas de reconocimiento y exportación para iOS

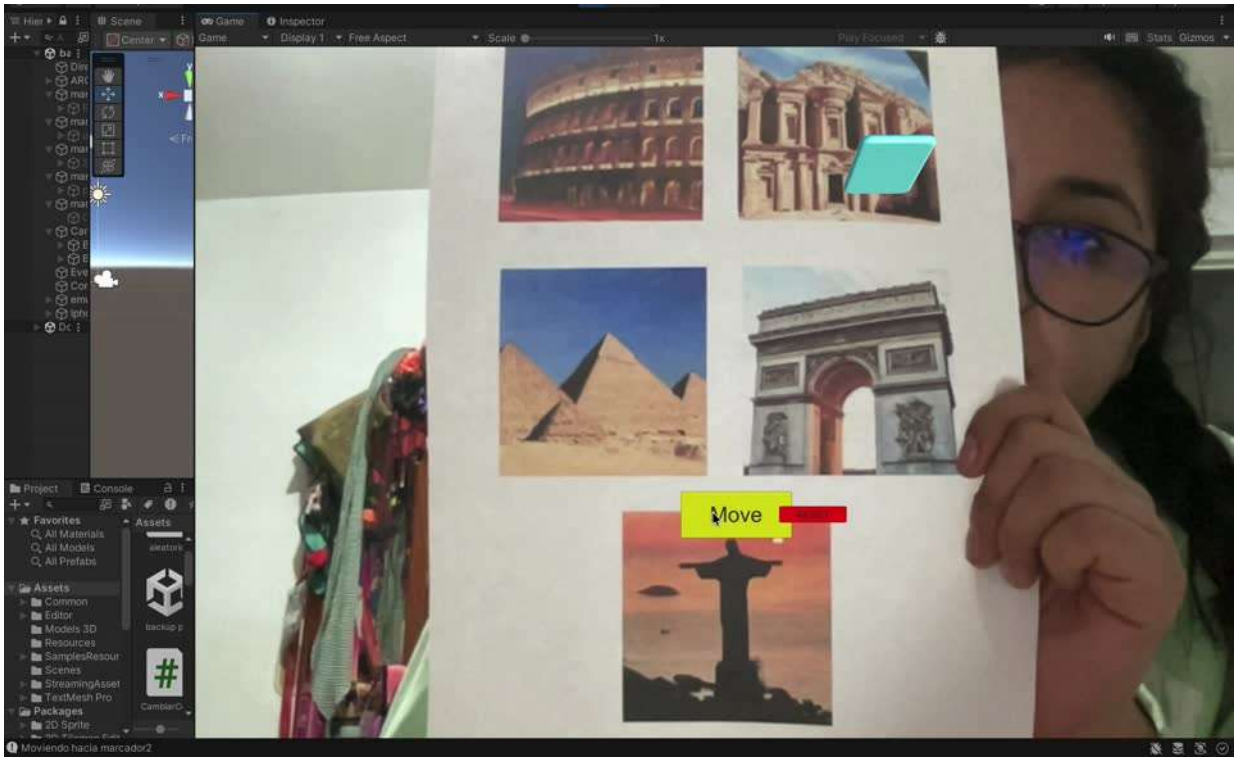


Figura 32. Prueba de marcadores ante cámara

La hoja contiene las cinco imágenes de monumentos y lugares históricos. Se muestra Game con controles MOVE y RESET sobre la captura de cámara, evidenciando la preparación de una prueba de reconocimiento múltiple.

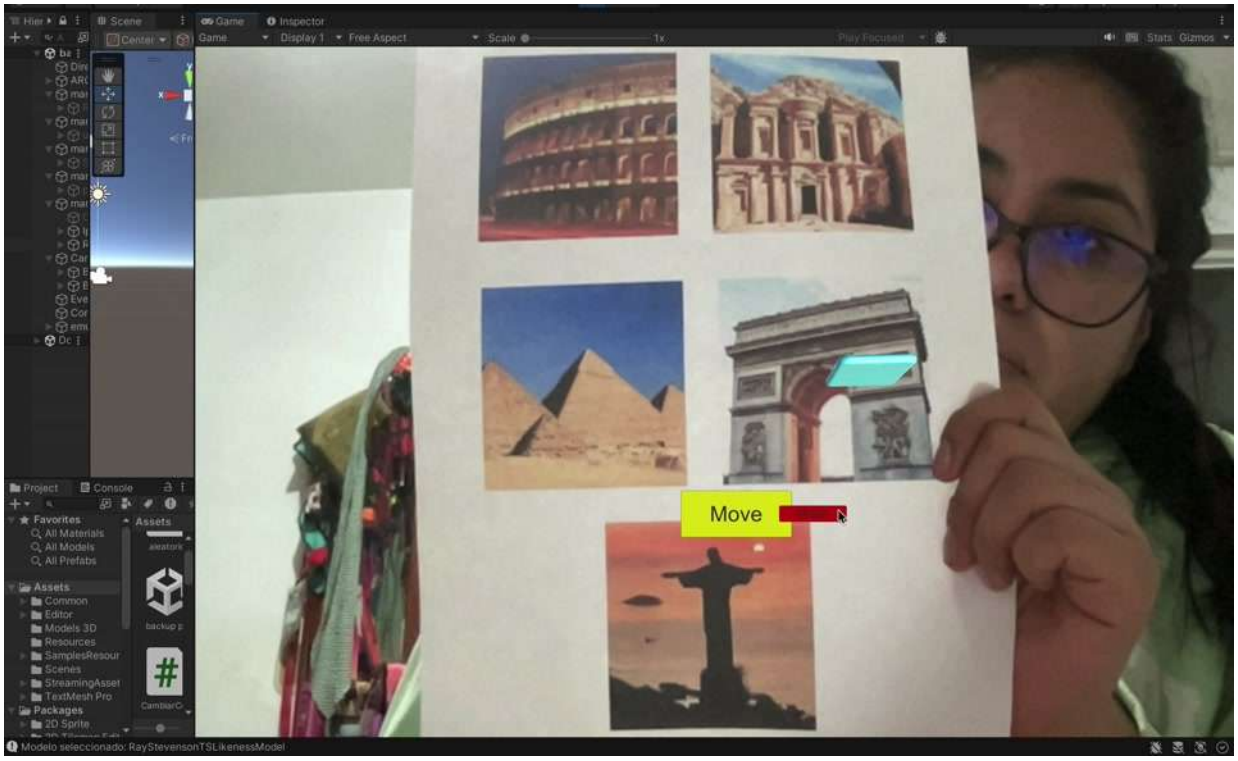


Figura 33. Prueba adicional con varios marcadores

La imagen muestra el seguimiento con varias referencias disponibles. Según el historial de pruebas, el marcador4 inicialmente no era seleccionado en determinadas condiciones; al aislar la imagen de las pirámides se logró comprobar reconocimiento.

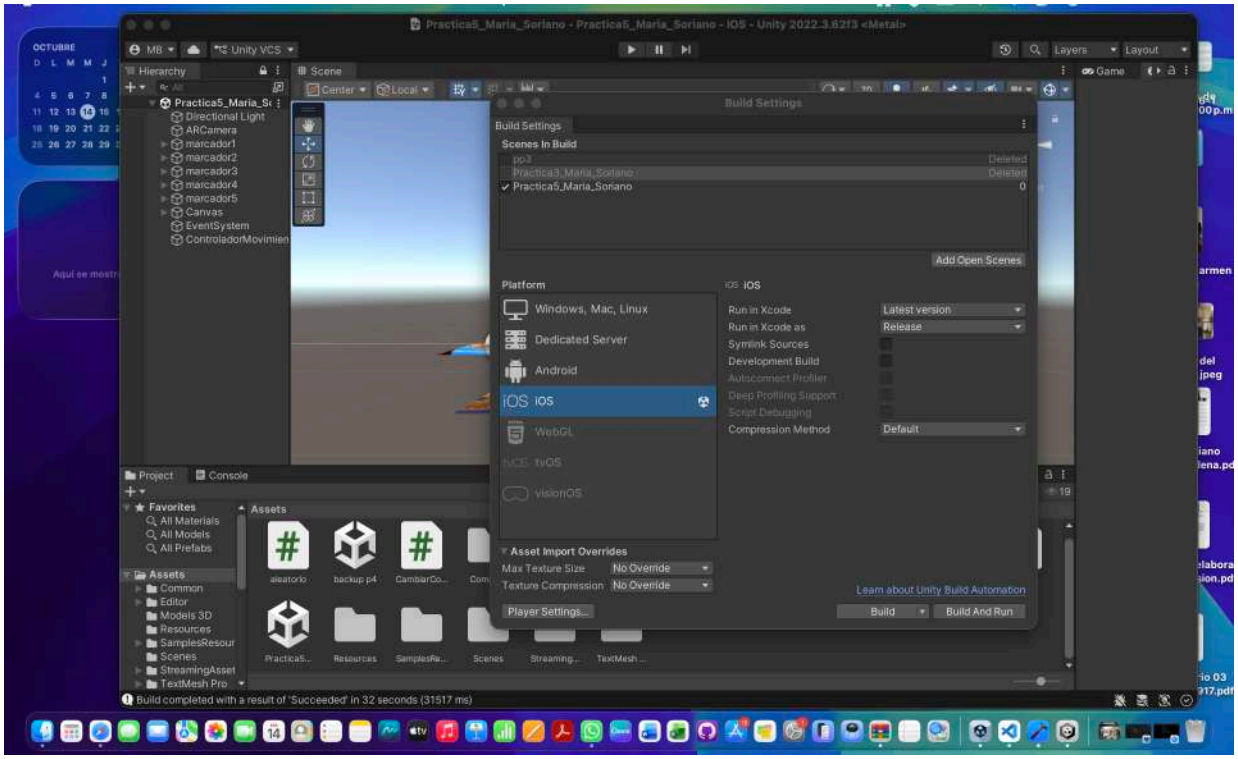


Figura 34. Exportación iOS desde Unity

Build Settings tiene iOS como plataforma y permite generar el proyecto de Xcode. La compilación para un iPhone se realiza después mediante el entorno de Apple.

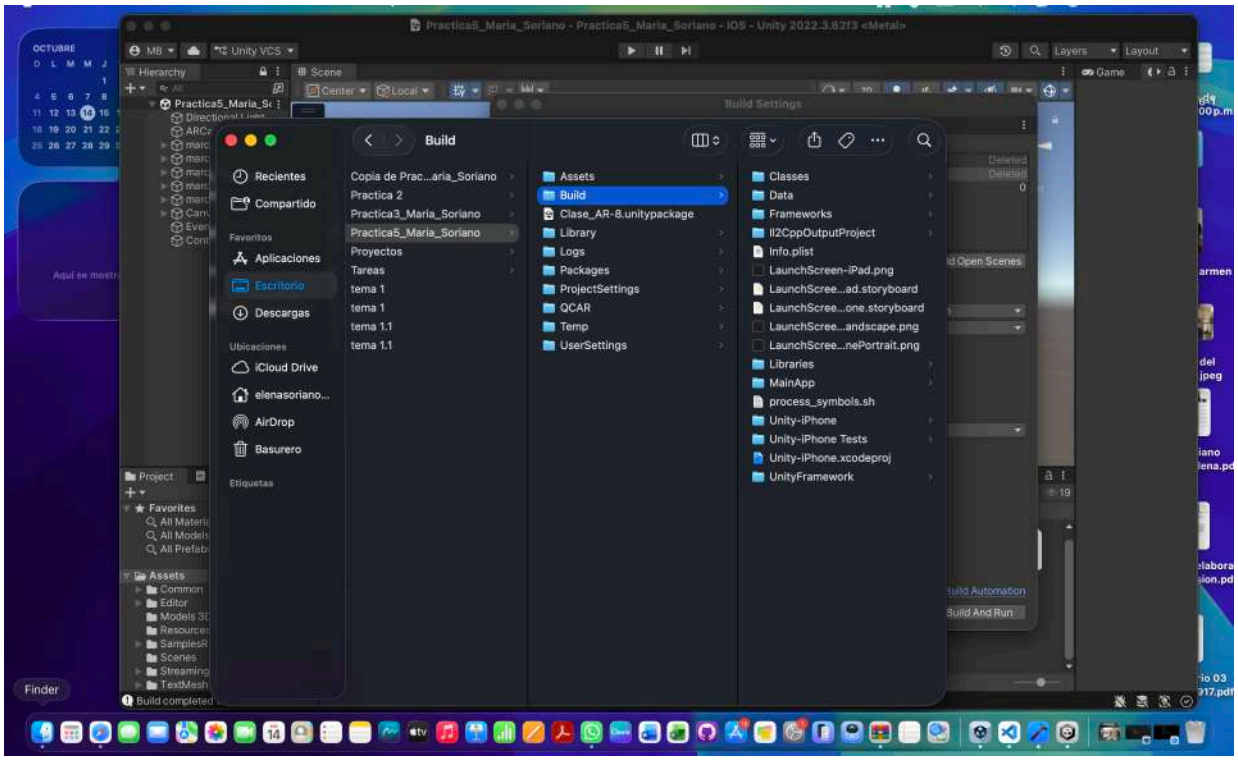


Figura 35. Selección de carpeta de salida

Finder registra la ubicación de destino del Build, donde Unity deposita el proyecto nativo que luego se compila y firma en Xcode.

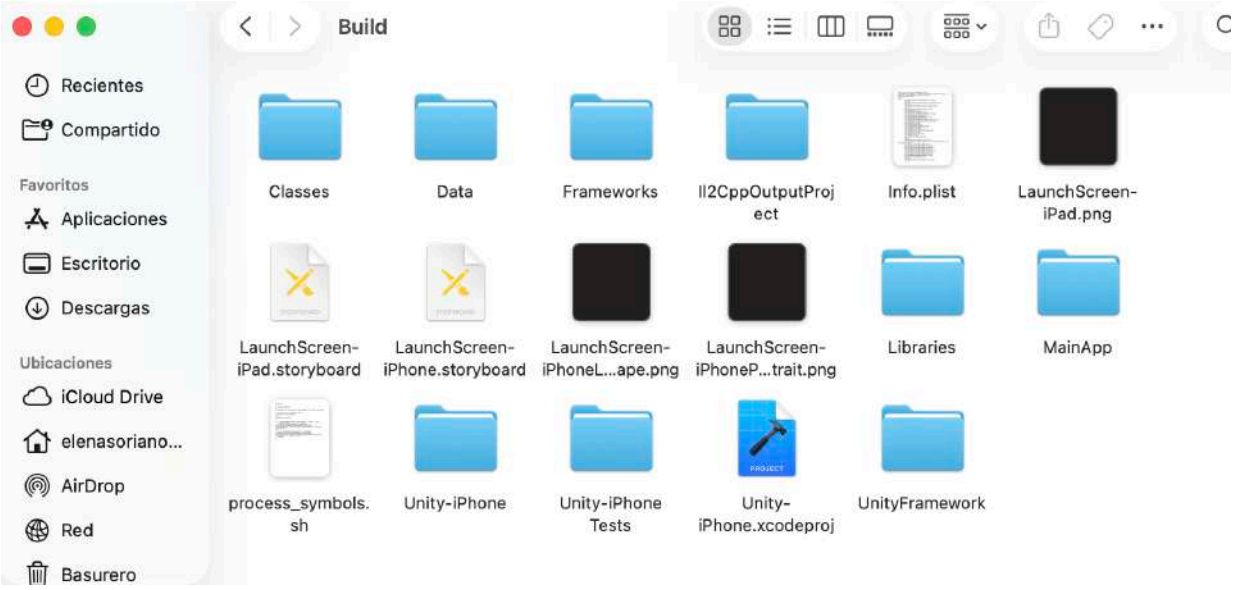


Figura 36. Estructura final del proyecto generado

Se vuelve a observar la carpeta de exportación con Unity-iPhone.xcodeproj; debe conservarse junto con los demás archivos del Build al abrir Xcode.

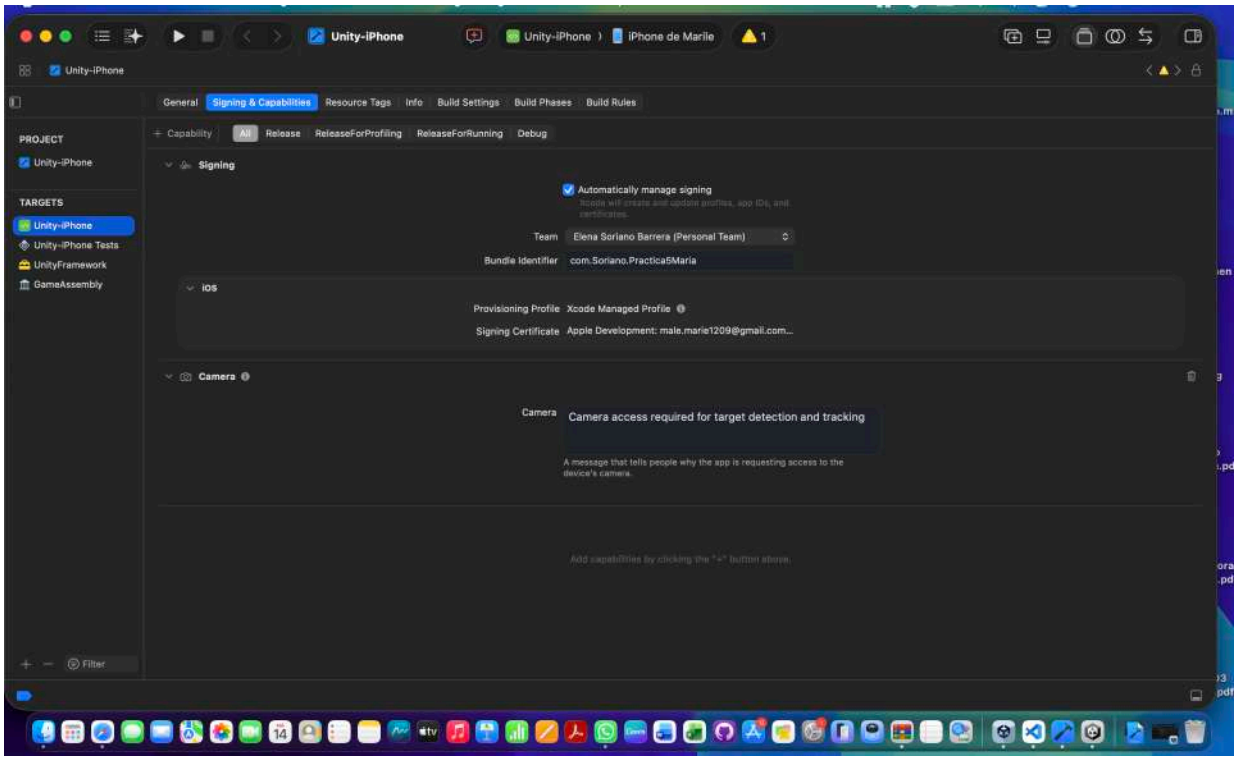


Figura 37. Configuración final en Xcode

La captura muestra el target Unity-iPhone y un dispositivo físico seleccionado. No basta para afirmar que la aplicación quedó instalada o que se verificó un recorrido completo en el teléfono.

6. Resultados

Las evidencias del proyecto permiten describir la configuración de cinco marcadores, integración de modelos OBJ, interfaz de bienvenida y controles, movimiento interpolado del iPhone, selección de un objeto alternativo y actualización de mensajes narrativos. Las capturas de Game muestran pruebas con imágenes impresas y representación de objetos tridimensionales sobre ellas.

Se resolvió una incidencia de seguimiento: marcador4 podía reconocerse al probarlo de forma aislada, aunque no siempre era elegido en presencia de varias imágenes. Esto indica que la disponibilidad efectiva del objetivo es condición para MOVE y no demuestra un error único en el código.

La separación de elementos se controla con separacionIphone y posicionAparicion; estos valores desplazan puntos de anclaje, pero no normalizan automáticamente la geometría de los cinco objetos. Las diferencias de tamaño entre modelos pueden requerir ajustes individuales de escala.

7. Evidencia en video

Video final de demostración se encuentra en el siguiente link:
https://youtube.com/shorts/YPx9PnwqZ4U?si=MfmCO7-Hqpzpf2A0

8. Repositorio de GitHub

Repositorio de la materia: https://github.com/mariaelena-soriano/TSIC3_RealidadAumentada

Enlace directo a la Práctica 5: https://github.com/mariaelena-soriano/TSIC3_RealidadAumentada/tree/main/Practica5_MariaSoriano

9. Conclusiones

La práctica permitió integrar reconocimiento de imágenes con una secuencia interactiva orientada a la exploración de lugares históricos. La división en tres scripts separó el flujo de inicio, el movimiento dependiente del seguimiento y la elección de una misión aleatoria. Esta organización permitió depurar problemas de detección y presentación sin reemplazar toda la escena.

Las pruebas resaltaron la necesidad de revisar las referencias del Inspector y de distinguir entre objetos existentes en Scene y objetivos que Vuforia está rastreando realmente. La exportación iOS y la firma en Xcode dejaron preparada la siguiente etapa de validación en un dispositivo físico. Para una entrega completa deben acreditarse con evidencia los criterios todavía no comprobados, especialmente la restricción a dos marcadores simultáneos y las variaciones de personaje.

11. Referencias

Unity Technologies. (2023). Unity User Manual 2022.3 LTS. https://docs.unity3d.com/2022.3/Documentation/Manual/

Unity Technologies. (2023). Coroutines. Unity Manual. https://docs.unity3d.com/2022.3/Documentation/Manual/Coroutines.html

Unity Technologies. (2023). Vector3.Lerp. Unity Scripting API. https://docs.unity3d.com/2022.3/Documentation/ScriptReference/Vector3.Lerp.html

PTC. (s. f.). Vuforia Engine Developer Library. https://developer.vuforia.com/library/

Apple. (s. f.). Xcode documentation. https://developer.apple.com/documentation/xcode/

Anexo A. Códigos completos en C#

A.1 Código aleatorio.cs

Código transcrito del documento original para consulta y edición; se conserva su lógica, nombres de métodos y referencias.

```
using UnityEngine;
using TMPro;

public class aleatorio : MonoBehaviour
{
    [Header("Los 5 modelos 3D")]
    public GameObject[] objetos;

    [Header("Texto de la narrativa")]
    public TMP_Text textoMision;

    [Header("Posicion de aparicion")]
    public Vector3 posicionAparicion =
        new Vector3(0.15f, 0.05f, 0f);

    private int ultimoIndice = -1;
    private GameObject modeloActual;

    void Start()
    {
        MostrarMensaje(
            "Descubre tu proxima mision. " +
            "¡Presiona MOVE!"
        );
    }

    // ELEGIR Y MOSTRAR MODELO ALEATORIO
    public void MostrarAleatorio(Transform destino)
```

```
{
    if (objetos == null || objetos.Length == 0)
    {
        Debug.LogWarning("No hay modelos asignados.");
        return;
    }

    if (destino == null)
        return;

    OcultarModelo();

    int indice;

    do
    {
        indice = Random.Range(0, objetos.Length);
    }
    while (objetos.Length > 1 &&
           indice == ultimoIndice);

    GameObject original = objetos[indice];

    if (original == null)
        return;

    modeloActual = Instantiate(
        original,
        destino,
        false
    );

    modeloActual.name = original.name + "_Aleatorio";

    modeloActual.transform.localPosition =
        posicionAparicion;

    modeloActual.transform.localRotation =
        original.transform.localRotation;

    modeloActual.transform.localScale =
        original.transform.localScale;

    modeloActual.SetActive(true);

    ultimoIndice = indice;

    // Mostrar la mision correspondiente
    MostrarMensaje(ObtenerMision(indice));

    Debug.Log(
        "Modelo seleccionado: " + original.name
    );
}

// NARRATIVA DE PASAPORTE AR
private string ObtenerMision(int indice)
{
    switch (indice)
    {
        case 0:
            return "MISION: VISITA UN MUSEO\n" +
                   "Descubre una obra de arte " +
                   "o un personaje historico.";

        case 1:
            return "MISION: VIAJA EN AVION\n" +
                   "¡Preparate para descubrir " +
                   "tu siguiente destino!";

        case 2:
            return "MISION: EXPLORA EN TREN\n" +
                   "Recorre la ciudad y conoce " +
                   "un lugar historico.";

        case 3:
            return "MISION: DESCUBRE UN MONUMENTO\n" +
                   "Investiga un dato interesante " +
                   "sobre las piramides.";

        case 4:
            return "MISION: CAPTURA UN RECUERDO\n" +
                   "Toma una fotografia del " +
                   "monumento que estas visitando.";

        default:
            return "¡Nueva mision desbloqueada!";
    }
```

```
    }

    // ACTUALIZAR TEXTO DE LA INTERFAZ
    public void MostrarMensaje(string mensaje)
    {
        if (textoMision != null)
        {
            textoMision.text = mensaje;
        }
    }

    // OCULTAR MODELO ANTERIOR
    public void OcultarModelo()
    {
        if (modeloActual != null)
        {
            Destroy(modeloActual);
            modeloActual = null;
        }
    }

    // BOTON RESET
    public void ResetearAleatorio()
    {
        OcultarModelo();

        ultimoIndice = -1;

        MostrarMensaje(
            "¡Bienvenido a Pasaporte AR!\n" +
            "Presiona MOVE para descubrir " +
            "una nueva mision."
        );
    }
}
```

A.2 Código move.cs

Código transcrito del documento original para consulta y edición; se conserva su lógica, nombres de métodos y referencias.

```
using System.Collections;
using UnityEngine;
using Vuforia;

public class move : MonoBehaviour
{
    [Header("Modelo que se movera")]
    public GameObject model;

    [Header("Marcadores de Vuforia")]
    public ObserverBehaviour[] ImageTargets;

    [Header("Seleccion aleatoria")]
    public aleatorio selectorAleatorio;

    [Header("Configuracion del movimiento")]
    public int currentTarget = -1;
    public float speed = 1.0f;

    [Header("Separacion visual")]
    public float separacionIphone = -0.15f;

    private bool isMoving = false;

    private Transform padreInicial;
    private Vector3 posicionInicial;
    private Quaternion rotacionInicial;
    private Vector3 escalaInicial;

    private Transform marcadorActual;

    void Start()
    {
        if (model == null)
        {
            Debug.LogError("No se ha asignado el iPhone.");
            return;
        }

        padreInicial = model.transform.parent;
        marcadorActual = padreInicial;
```

```
        posicionInicial = model.transform.localPosition;
        rotacionInicial = model.transform.localRotation;
        escalaInicial = model.transform.localScale;

        if (selectorAleatorio == null)
        {
            selectorAleatorio = GetComponent<aleatorio>();
        }
    }

    // BOTON MOVE
    public void moveToNexMarker()
    {
        if (model == null || isMoving)
            return;

        ObserverBehaviour destino = BuscarSiguienteMarcador();

        if (destino == null)
        {
            Debug.LogWarning(
                "No hay otro marcador detectado."
            );
            return;
        }

        StartCoroutine(MoverIphone(destino));
    }

    private IEnumerator MoverIphone(ObserverBehaviour destino)
    {
        isMoving = true;

        Debug.Log("Iniciando movimiento hacia: " + destino.name);

        if (selectorAleatorio != null)
        {
            selectorAleatorio.OcultarModelo();
        }

        Vector3 inicio = model.transform.position;

        model.transform.SetParent(null, true);

        Vector3 offsetLocal =
            posicionInicial + new Vector3(separacionIphone, 0f, 0f);

        float duracion = Mathf.Max(
            0.1f,
            1f / Mathf.Max(speed, 0.01f)
        );

        float tiempo = 0f;

        while (tiempo < duracion)
        {
            if (destino == null ||
                !MarcadorDetectado(destino))
            {
                Debug.LogWarning(
                    "Se perdio el marcador durante el movimiento."
                );

                RestaurarMarcadorActual();
                isMoving = false;
                yield break;
            }

            tiempo += Time.deltaTime;

            float progreso = Mathf.Clamp01(
                tiempo / duracion
            );

            float suavizado = Mathf.SmoothStep(
                0f, 1f, progreso
            );

            Vector3 posicionDestino =
                destino.transform.TransformPoint(offsetLocal);

            model.transform.position = Vector3.Lerp(
                inicio,
                posicionDestino,
                suavizado
            );

            yield return null;
```

```
        }

        model.transform.SetParent(
            destino.transform,
            true
        );

        // Colocar el iPhone un poco a la izquierda
        model.transform.localPosition =
            posicionInicial + new Vector3(separacionIphone, 0f, 0f);

        model.transform.localRotation = rotacionInicial;
        model.transform.localScale = escalaInicial;

        marcadorActual = destino.transform;

        Debug.Log("El iPhone llego a: " + destino.name);

        if (selectorAleatorio != null)
        {
            selectorAleatorio.MostrarAleatorio(
                destino.transform
            );
        }

        isMoving = false;
    }

    // VERIFICAR MARCADORES
    private bool MarcadorDetectado(ObserverBehaviour target)
    {
        if (target == null)
            return false;

        Status estado = target.TargetStatus.Status;

        return estado == Status.TRACKED ||
               estado == Status.EXTENDED_TRACKED;
    }

    // BUSCAR SIGUIENTE MARCADOR
    private ObserverBehaviour BuscarSiguienteMarcador()
    {
        if (ImageTargets == null ||
            ImageTargets.Length == 0)
            return null;

        for (int i = 0; i < ImageTargets.Length; i++)
        {
            int indice = (currentTarget + i + 1)
                         % ImageTargets.Length;

            ObserverBehaviour target = ImageTargets[indice];

            if (target != null &&
                target.transform != marcadorActual &&
                MarcadorDetectado(target))
            {
                currentTarget = indice;
                return target;
            }
        }

        return null;
    }

    // RESTAURAR POSICION
    private void RestaurarMarcadorActual()
    {
        if (model == null)
            return;

        Transform padre = marcadorActual != null
            ? marcadorActual
            : padreInicial;

        model.transform.SetParent(padre, false);

        // Si vuelve al marcador inicial, posicion original
        if (padre == padreInicial)
        {
            model.transform.localPosition = posicionInicial;
        }
        else
        {
            model.transform.localPosition =
                posicionInicial + new Vector3(separacionIphone, 0f, 0f);
        }
```

```
        model.transform.localRotation = rotacionInicial;
        model.transform.localScale = escalaInicial;
    }

    // BOTON RESET
    public void Resetear()
    {
        StopAllCoroutines();

        isMoving = false;
        currentTarget = -1;

        if (selectorAleatorio != null)
        {
            selectorAleatorio.ResetearAleatorio();
        }

        marcadorActual = padreInicial;

        if (model != null)
        {
            model.transform.SetParent(
                padreInicial,
                false
            );

            model.transform.localPosition = posicionInicial;
            model.transform.localRotation = rotacionInicial;
            model.transform.localScale = escalaInicial;
        }

        Debug.Log("Experiencia reiniciada.");
    }
}
```

A.3 Código InicioPasaporte.cs

Código transcrito del documento original para consulta y edición; se conserva su lógica, nombres de métodos y referencias.

```
using UnityEngine;

public class InicioPasaporte : MonoBehaviour
{
    public GameObject panelBienvenida;
    public GameObject controlesAR;

    void Start()
    {
        if (panelBienvenida != null)
            panelBienvenida.SetActive(true);

        if (controlesAR != null)
            controlesAR.SetActive(false);
    }

    public void Comenzar()
    {
        if (panelBienvenida != null)
            panelBienvenida.SetActive(false);

        if (controlesAR != null)
            controlesAR.SetActive(true);
    }
}
```